CC7711

Inteligência Artificial e Robótica

Prof. Dr. Flavio Tonidandel

Redes Neurais Artificiais - Introdução

CC7711 - Inteligência Artificial e Robótica

Redes Neurais Artificiais (RNA)

- Definição:
  - É um sistema de processamento de informações desenvolvido a partir de modelos matemáticos simples dos neurônios biológicos.
  - É constituída por um grande número de unidades de processamento (chamadas neurônios artificiais) interconectadas.
  - Reconhecem, classificam, convertem e aprendem padrões.

- Idéia central:
  - utilizar neurônios biológicos como modelos para neurônios artificiais.

Nosso Cérebro:

- Rede de neurônios:
- 10^{11} neurônios conectados a aprox. 10^{4} outros
- Tempo para um neurônio processar a informação: 10^{-3}
  - Computador: 10^{-10}
- Levamos surpreendentes 10^{-1} segundos para reconhecer o rosto de nossa mãe.
  - Poucas centenas de neurônios em seqüência
  - O cérebro tem que ser altamente paralelo.

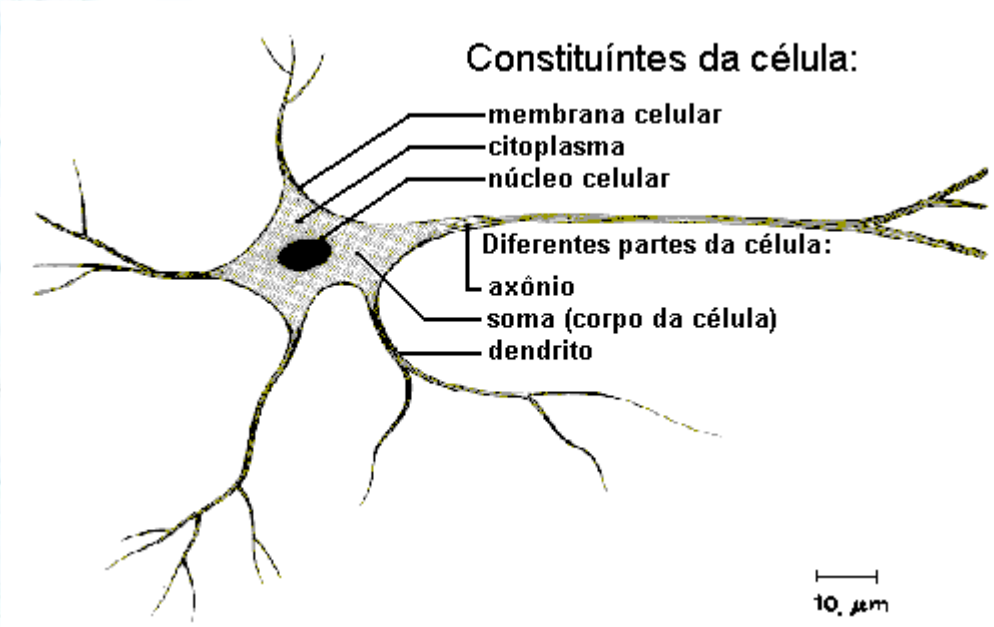
- Os neurônios do cérebro possui o seguinte funcionamento...

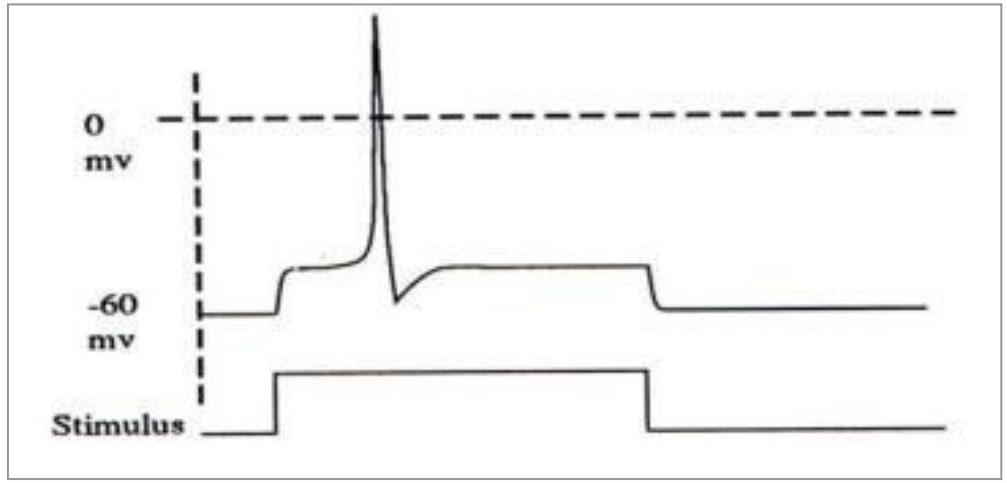
Neurônio Biológico

- Elemento fundamental do sistema nervoso.
- Diversos tipos.

Recebe estímulos pelos dendritos, e emite estímulos pelo axônio

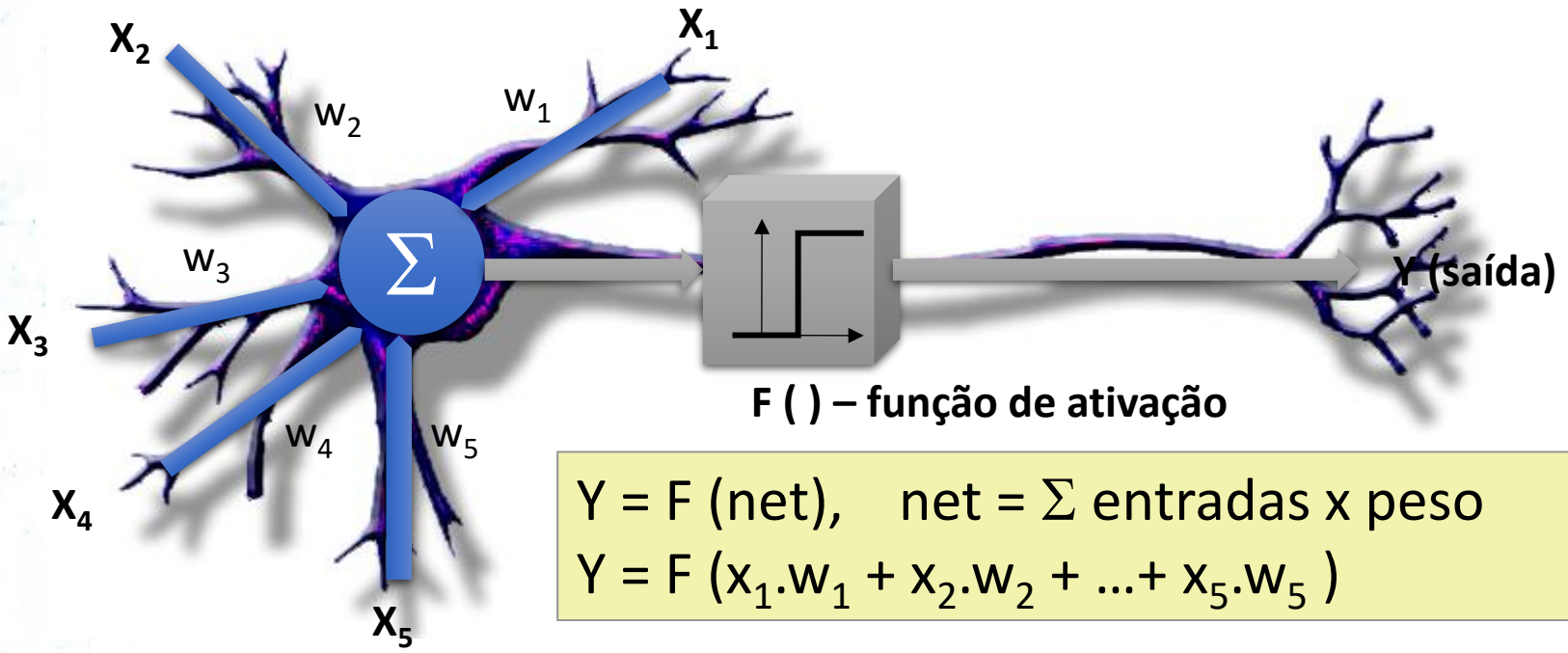
Tudo isso é feito quimicamente por diferentes concentrações de Na+ (Sódio) e K+ (potássio).





Modelo de um Neurônio (McCulloch-Pitts)

Modelo matemático

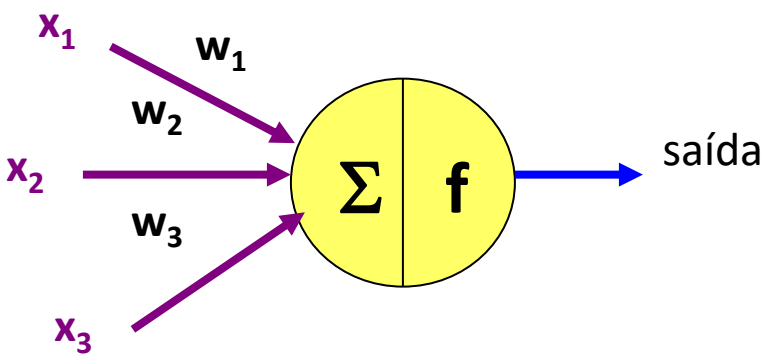


$Y = F\,(net)$, net = Σ entradas x peso

$Y = F\,(x_1.w_1 + x_2.w_2 + \ldots + x_5.w_5\,)$

Modelo de um Neurônio (McCulloch-Pitts)

Equação Geral:



w =Vetor de pesos.
x = Vetor de entradas.
Σ = Soma ponderada das entradas
f = função de ativação.

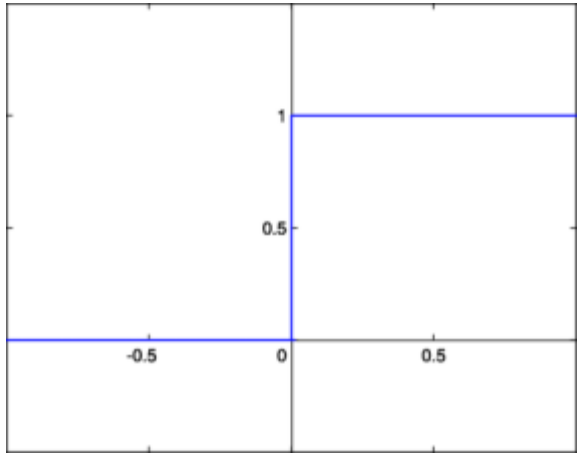
$$\text{saída} = y = f\,[(w_1.x_1)+(w_2.x_2)+(w_3.x_3)]$$

Funções de Ativação

Modelo McCulloch-Pitts

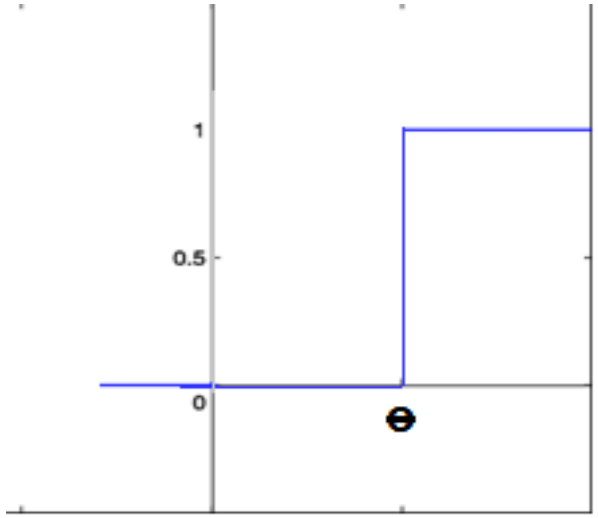
- Hard limit (degrau)
  - Saída = 1 se net $\geq$ 0

    = 0 se net < 0



- Alternativa:
- Função degrau com limiar
  - Define-se um valor θ
  - Saída = 1 se net $\geq \theta$

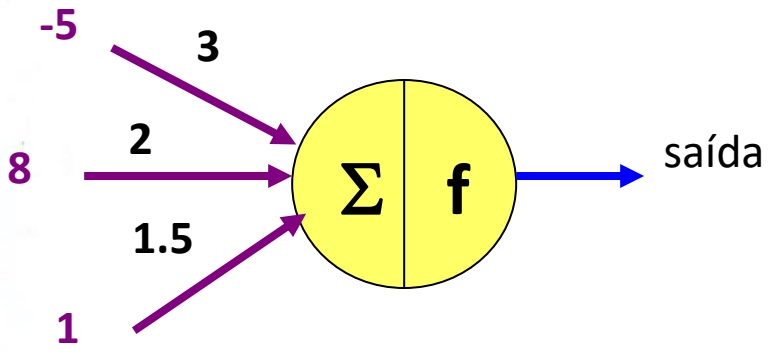
    = 0 se net < θ

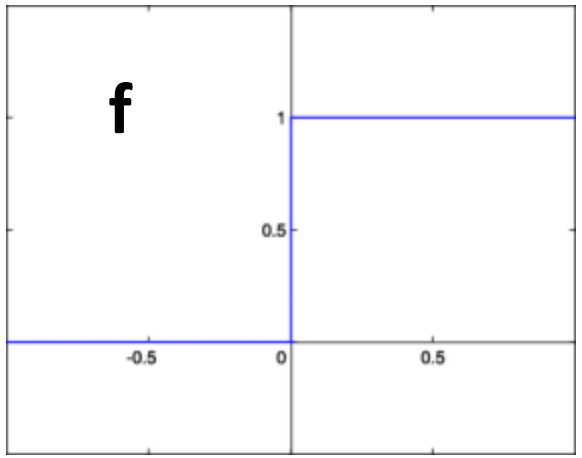


Imagens em licença CC

Procedimento de Recall

Recall é o processo de calcular a saída de uma rede neural dado um vetor de entrada.





saída = y = f [net] = f $[(w_1.x_1)+(w_2.x_2)+(w_3.x_3)]$

saída = y = f [(3*-5)+(2*8)+(1,5*1)] = f [-15+16+1,5]

saída = y = f [2,5] = 1

Perspectivas Históricas

1943 - Neurônio de McCulloch-Pitts

1959 – Rede Perceptrons (Rosenblatt) - prova de sua convergência

1960 - Adaline (Widrow & Hoff) – Primeira Rede Neural aplicada em problemas reais

1969 - Minsky and Papert (MIT) publicam um livro que provam a limitação dos perceptrons e redes como a Adaline.

1970-1981 – Pesquisas focadas no aprimoramento das redes neurais

1986 - Surgimento do algoritmo de retro-propagação (Rumelhart & McClelland) – Permitiu o desenvolvimento de redes mais sofisticadas e sem a limitação dos perceptrons

1987 - Redes de Hopfield (Hopfield & Tank)

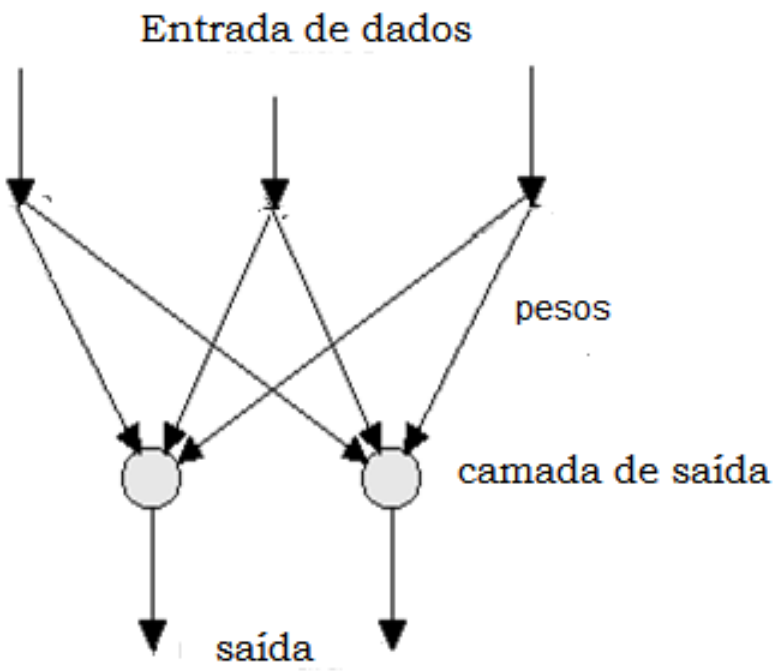
1990→hoje – amplamente utilizada em diversas aplicações.

Perceptron

Rede de neurônios McCulloch-Pitts

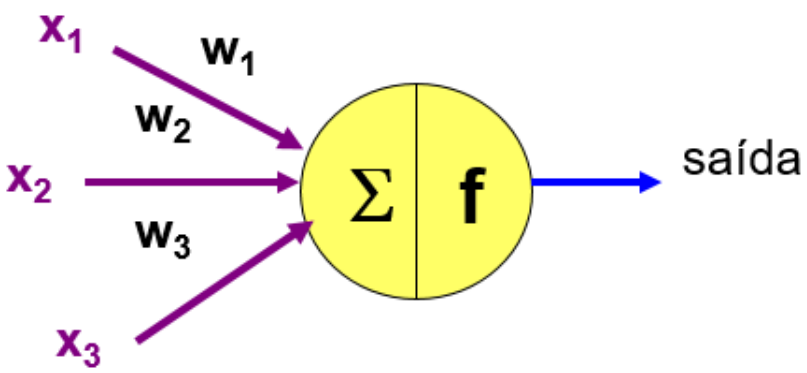
Criada por Rosenblatt em 1959

- Rede com neurônios formando a camada de Saída interligados com as entradas



Treinamento de um Perceptron

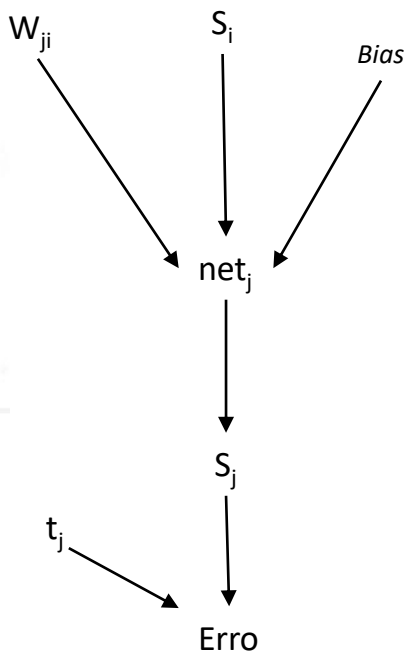
Baseado no erro da saída da rede neural

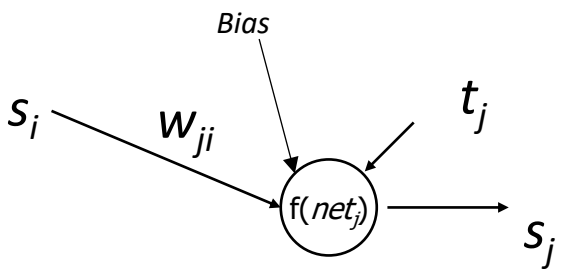


- Medida clássica: soma dos erros quadráticos
  - Erro = E = ½ $(Err)^2$ = ½ $(d-y)^2$;
  - d = saída desejada e y a saída real

- Podemos usar o Declínio Gradiente para reduzir o erro quadrático.
  - Calcula-se a derivada parcial de E em relação a cada peso
  - Como determinar isso ?

Determinando o quanto a variação de um peso influencia o Erro





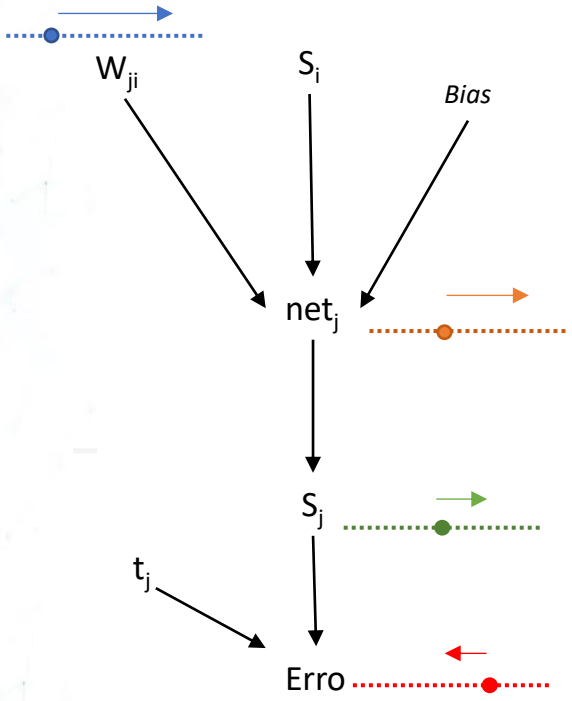
$$Erro = \tfrac{1}{2}(t_j - S_j)^2$$

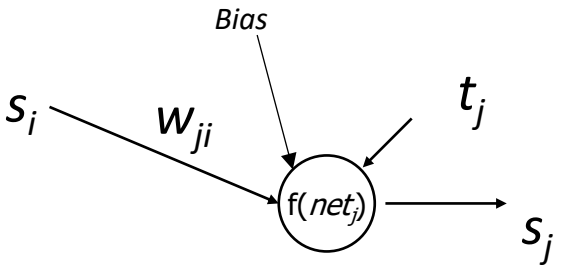
$$S_j = f(net_j) = f(Si*W_{ji} + Bias)$$

$$net_j = S_i*W_{ji} + Bias$$

Determinando o quanto a variação de um peso influencia o Erro

$$\frac{\partial Erro}{\partial W} = ?????$$ Quanto o erro altera com a variação de *W* ?





$$Erro = \tfrac{1}{2}(t_j - S_j)^2$$

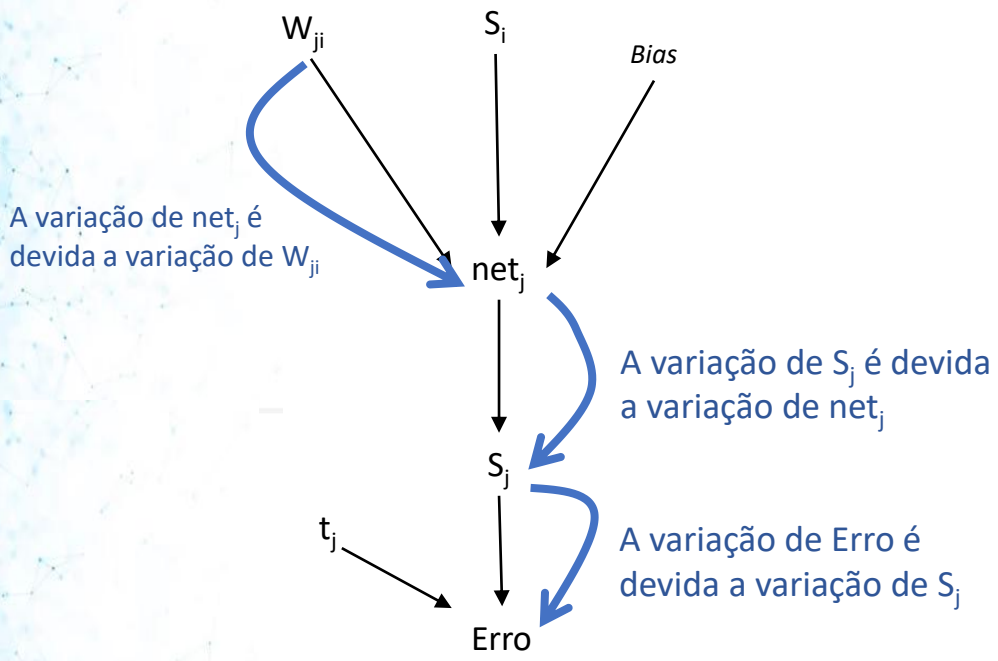
$$S_j = f(net_j) = f(Si*W_{ji} + Bias)$$

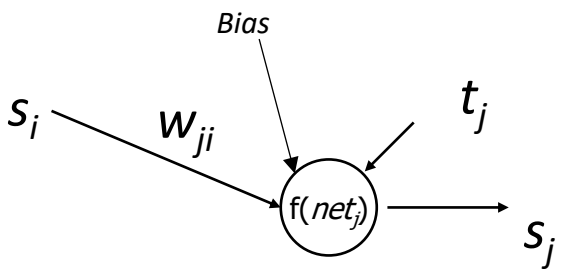
$$net_j = S_i*W_{ji} + Bias$$

Determinando o quanto a variação de um peso influencia o Erro

Regra da Cadeia

$$\frac{\partial Erro}{\partial W} = \frac{\partial net_j}{\partial W_{ji}} * \frac{\partial S_j}{\partial net_j} * \frac{\partial Erro}{\partial S_j}$$





$$Erro = ½(t_j - S_j)^2$$

$$S_j = f(net_j) = f(Si*W_{ji} + Bias)$$

$$net_j = S_i*W_{ji} + Bias$$

Determinando o quanto a variação de um peso influencia o Erro

Regra da Cadeia

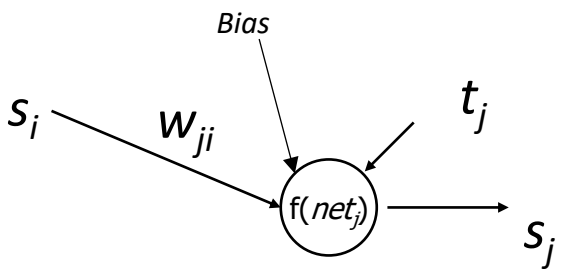
$$\frac{\partial Erro}{\partial W} = \frac{\partial net_j}{\partial W_{ji}} * \frac{\partial S_j}{\partial net_j} * \frac{\partial Erro}{\partial S_j}$$

S_i $\quad$ $f'(net_j)$ $\quad$ $(t_j - S_j)$

$$\frac{\partial net_j}{\partial W_{ji}} = \frac{\partial(Si * Wji + Bias)}{\partial W_{ji}} = Si$$

$$\frac{\partial S_j}{\partial net_j} = \frac{\partial(f(net_j))}{\partial net_j} = f'(netj)$$

$$\frac{\partial Erro}{\partial S_j} = \frac{\partial(½(tj - Sj)^2)}{\partial S_j} = (t_j - Sj)$$



$Erro = ½(t_j - S_j)^2$

$S_j = f(net_j) = f(Si*W_{ji} + Bias)$

$net_j = S_i*W_{ji} + Bias$

Determinando o quanto a variação de um peso influencia o Erro

Regra da Cadeia

$$\frac{\partial Erro}{\partial W} = \frac{\partial net_j}{\partial W_{ji}} * \frac{\partial S_j}{\partial net_j} * \frac{\partial Erro}{\partial S_j}$$

S_i $\quad$ $f'(net_j)$ $\quad$ $(t_j - S_j)$

Deste modo, a atualização do Peso se dá:

$$\Delta W = \eta \frac{\partial Erro}{\partial W}$$

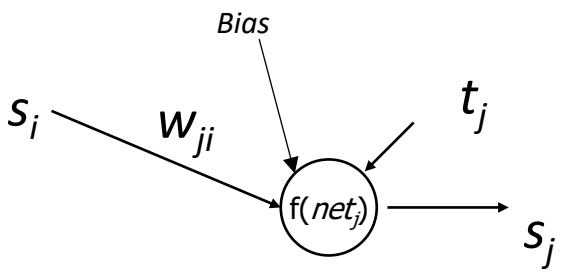
$\Delta W_{ji} = \eta * (tj - Sj) * f'(net_j) * S_i$

Ou

$Wj = Wj + \eta * (d - y) * f'(net) * xj$

d = tj = saída desejada

Y = Sj = Saída real da RN



$Erro = ½(t_j - S_j)^2$

$S_j = f(net_j) = f(Si*W_{ji} + Bias)$

$net_j = S_i*W_{ji} + Bias$

Treinamento de um Perceptron

- A atualização de um peso se dá por:

$$Wj = Wj + \eta \frac{\partial E}{\partial Wj}$$

$$Wj = Wj + \eta * (d - y) * f'(net) * xj$$

η é uma constante pequena (ex: 0.1), chamada taxa de aprendizado.

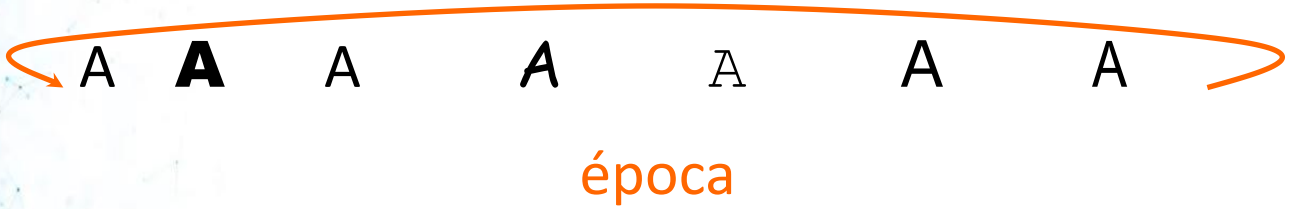
- *Para um perceptron, que usa função degrau, a derivada de f'(net) é 1. Logo:*

$$Wj = Wj + \eta * (d - y) * xj$$

Como treinar uma rede ?

- Convergência de uma rede
  - A rede é treinada usando-se alguns exemplos de treinamento.
  - Cada exemplo é aplicado na rede, resultando em ajustes dos pesos para reduzir o erro.
  - Os exemplos são repetidos em ciclos várias e várias vezes, até não houver nenhum erro para TODOS os exemplos.
  - Esses ciclos são chamados de época.

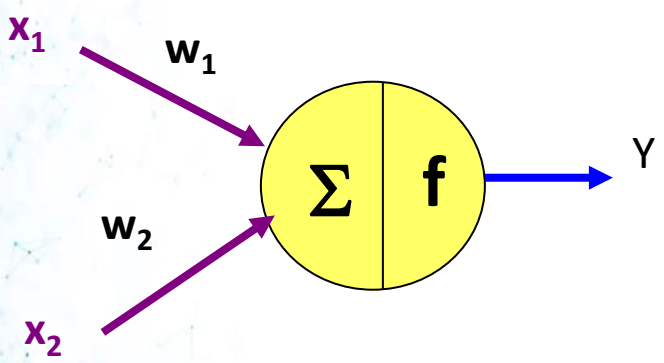
- Exemplo de treinamento de uma rede para letra A



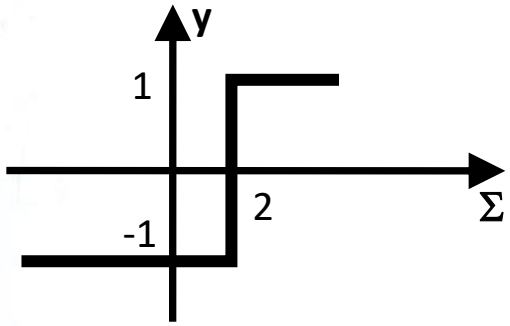

```
000001100000
000110011000
001100001100
011111111110
110000000011
110000000011
110000000011
```

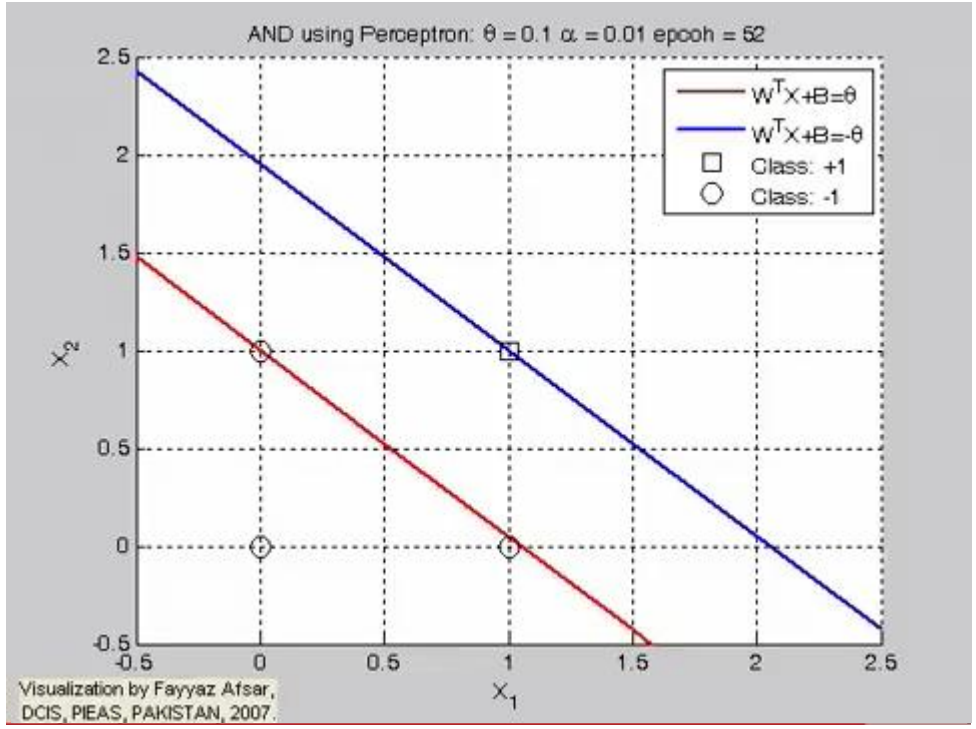
Imagem 12 x 7 = 84

Um Perceptron: Relação Lógica AND



y = 1 se nível de ativação ≥ 2,
-1 caso contrário





Rede Perceptron

- Minsky and Papert provaram que os Perceptrons são limitados
  - Só separam conjuntos linearmente separáveis

Embora limitado, possui a vantagem de ter um sistema de aprendizado simples e que converge para qualquer conjunto de treinamento linearmente separável

Ex: todas as funções booleanas primitivas (AND, OR, NAND e NOR).

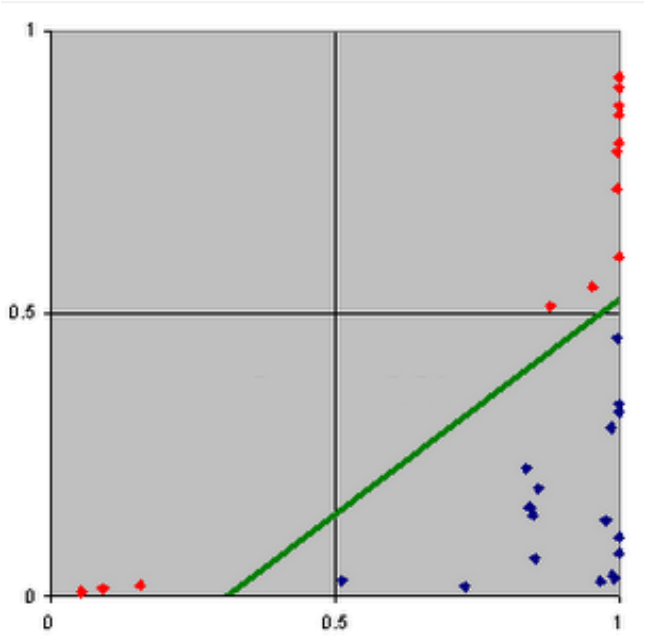
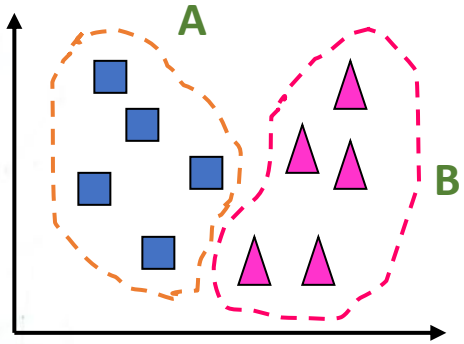
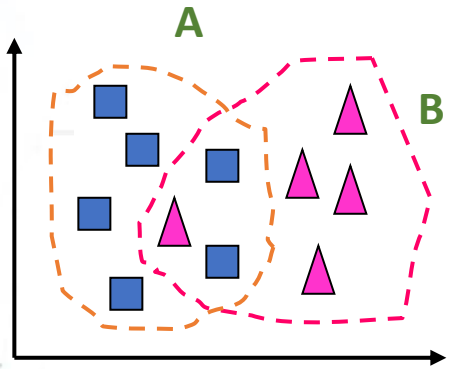


Imagem de John Wakefield (dynamic notion website)

O que é ser não linearmente separável ?



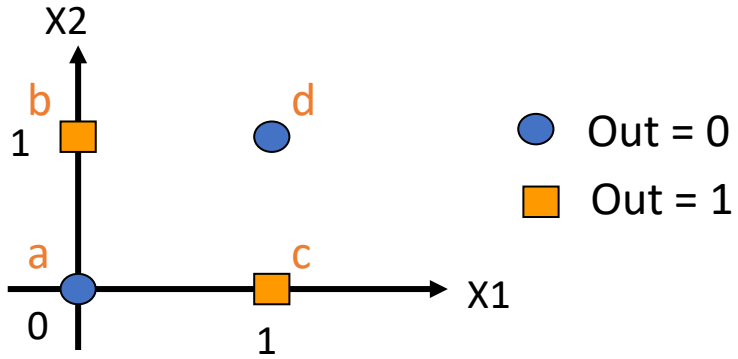
Linearmente separável



Não Linearmente separável

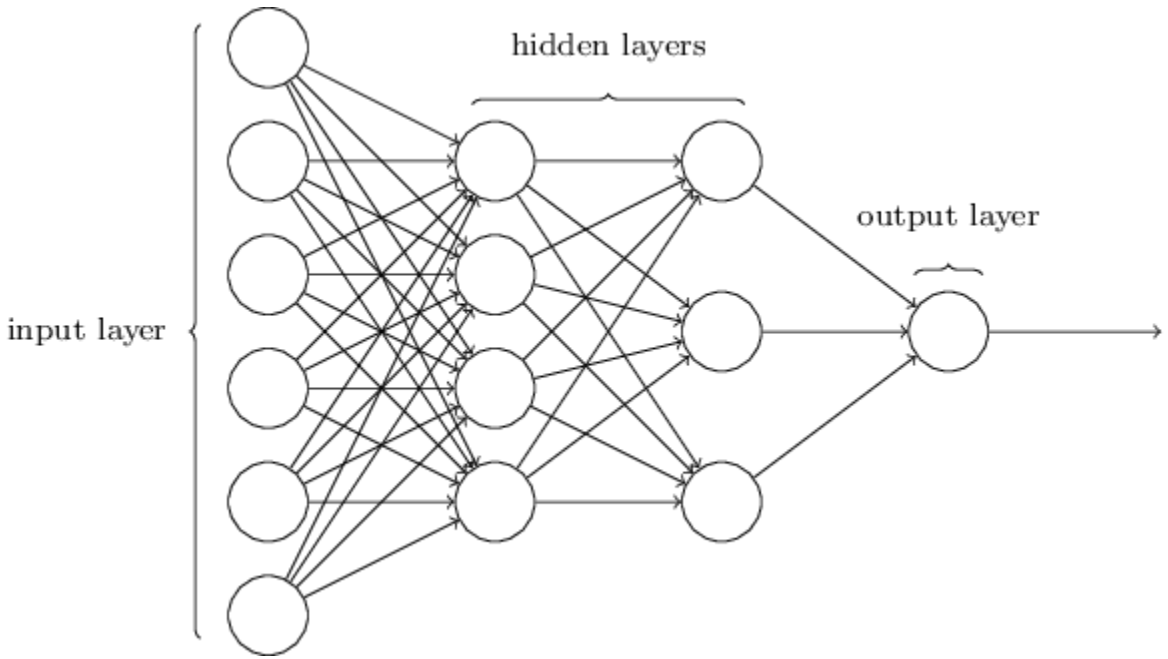
- O *Perceptron* não pode representar o XOR (ou exclusivo).

	X1	X2	Out
a	0	0	0
b	0	1	1
c	1	0	1
d	1	1	0



MLP: Multi Layer Perceptrons

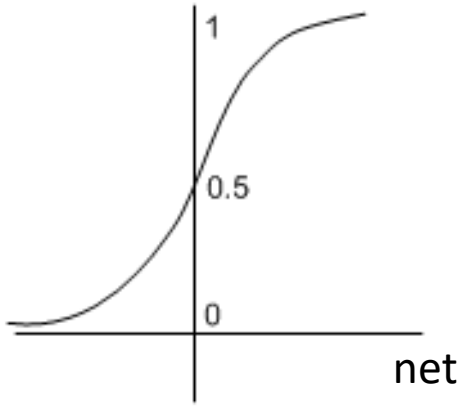
- Conjuntos de *Perceptons* com função de ativação sigmóide arranjados em diversas camadas.
- Possui pelo menos uma camada escondida.
- Soluciona o problema de classificação não linear pois consegue formar regiões de decisão mais complexas
- Cada camada é composta de uma rede linear.



Funções de Ativação Sigmóide

- Sendo *net* a soma ponderada da entrada, a saída Y será:

$$Y = \frac{1}{1+e^{-net}}$$

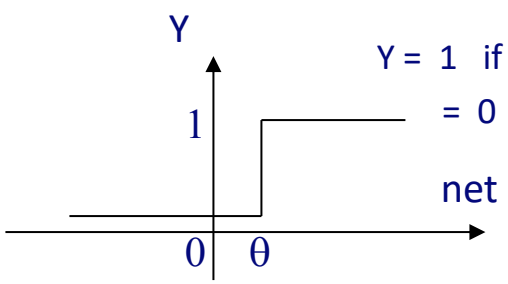


- Vantagens:
  - Se parece muito com a função degrau
  - A Derivada é facilmente expressa em termos da própria função Y:

$$\frac{dy}{d_{net}} = y\,(1 - y)$$

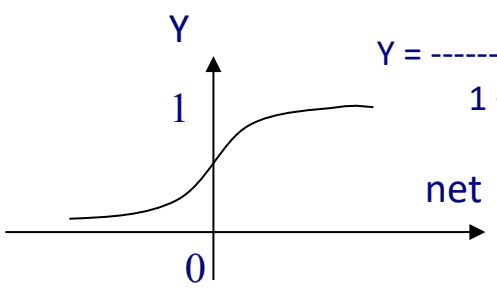
Variações da Função de Ativação

Função limiar- Degrau

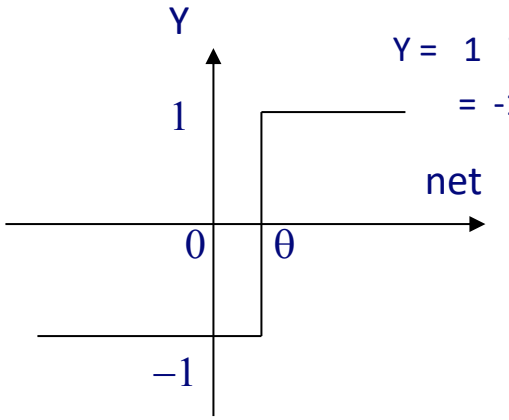


$$Y = 1 \text{ if } net > \theta$$
$$= 0 \text{ if } net < \theta$$

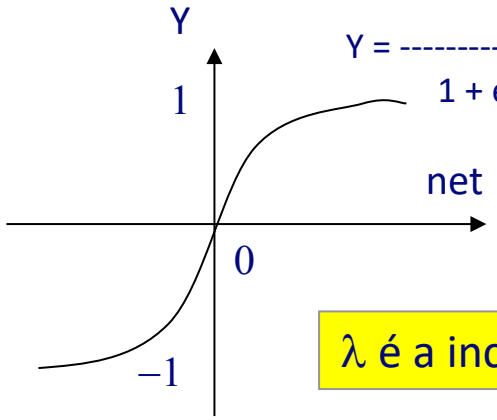
Função Sigmóide



$$Y = \frac{1}{1 + e^{(-\lambda.net)}}$$



$$Y = 1 \text{ if } net > \theta$$
$$= -1 \text{ if } net < \theta$$



$$Y = \frac{2}{1 + e^{(-\lambda.net)}} - 1$$

λ é a inclinação

MLP: Função Degrau ou Sigmóide ?

- Para conseguir que a MLP aproxime funções não-lineares, suas camadas (ou pelo menos algumas delas) deve possuir função sigmóide.

- O uso da função degrau em toda a MLP irá torná-la um aproximador de funções lineares.

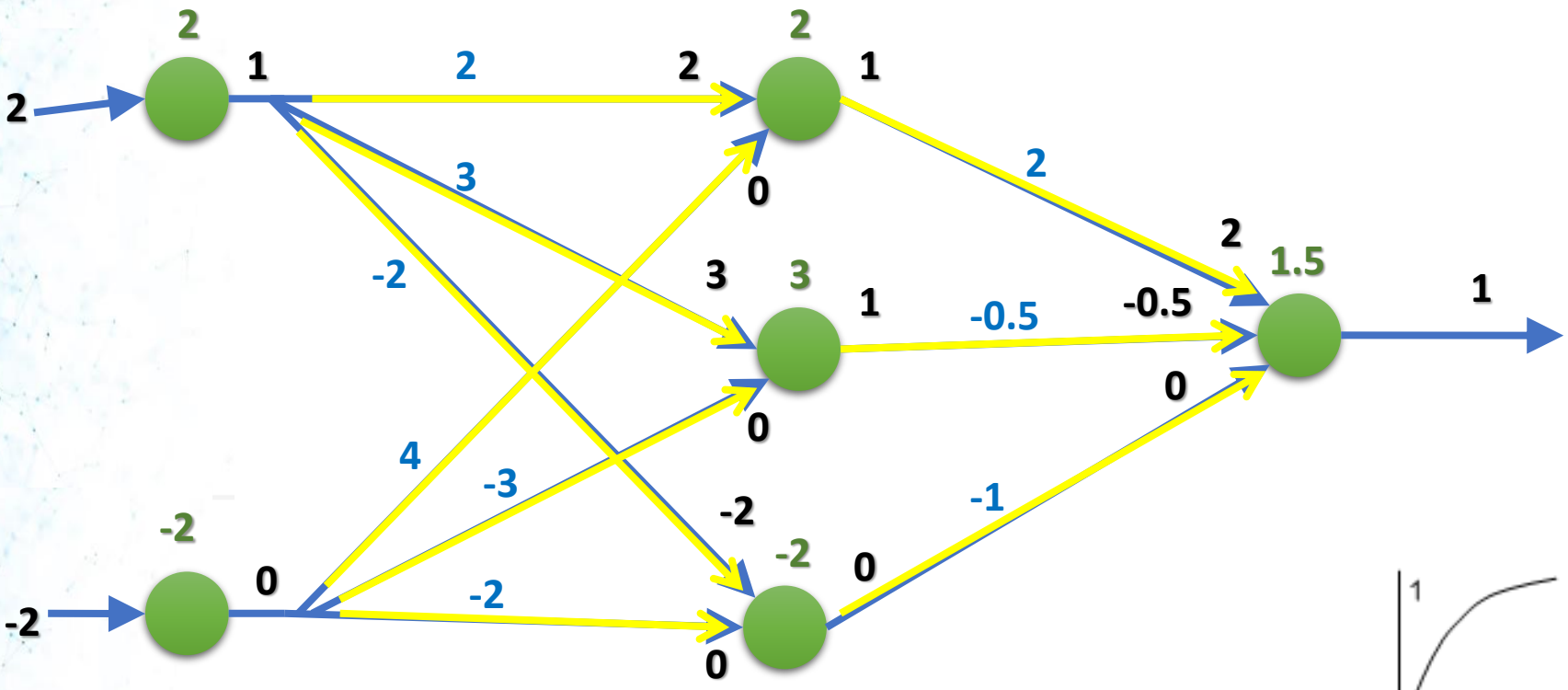
- Qualquer função booleana pode ser representada por uma RNA com uma camada escondida, usando funções degrau.

- Qualquer função contínua, linear ou não linear, pode ser aproximada por uma RNA com duas camadas escondidas, usando funções sigmóides.

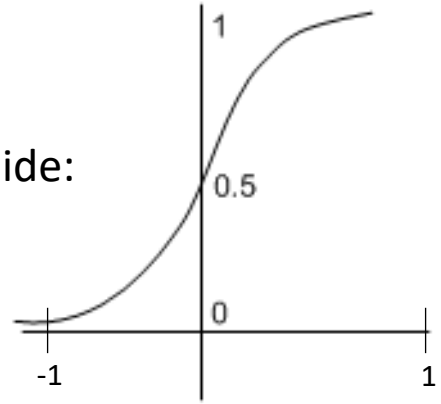
Construção de uma RNA (Rede Neural Artificial)

- Etapas:
  - Definir neurônios: função de ativação.
  - Definir conexões: pesos, ordem, topologia (conectividade).
  - procedimento de aprendizado.
    - O aprendizado consiste em modificar o peso das conexões entre os neurônios para que a rede faça o processamento desejado
- Características:
  - Os neurônios de uma RNAs devem estar conectados entre si.
  - Os neurônios são dispostos em camadas:
    - neurônios de uma mesma camada se comportam da mesma maneira.
  - A disposição dos neurônios nas camadas e o padrão de conexão entre camadas define a Arquitetura da RNA.

MLP em ação !!!

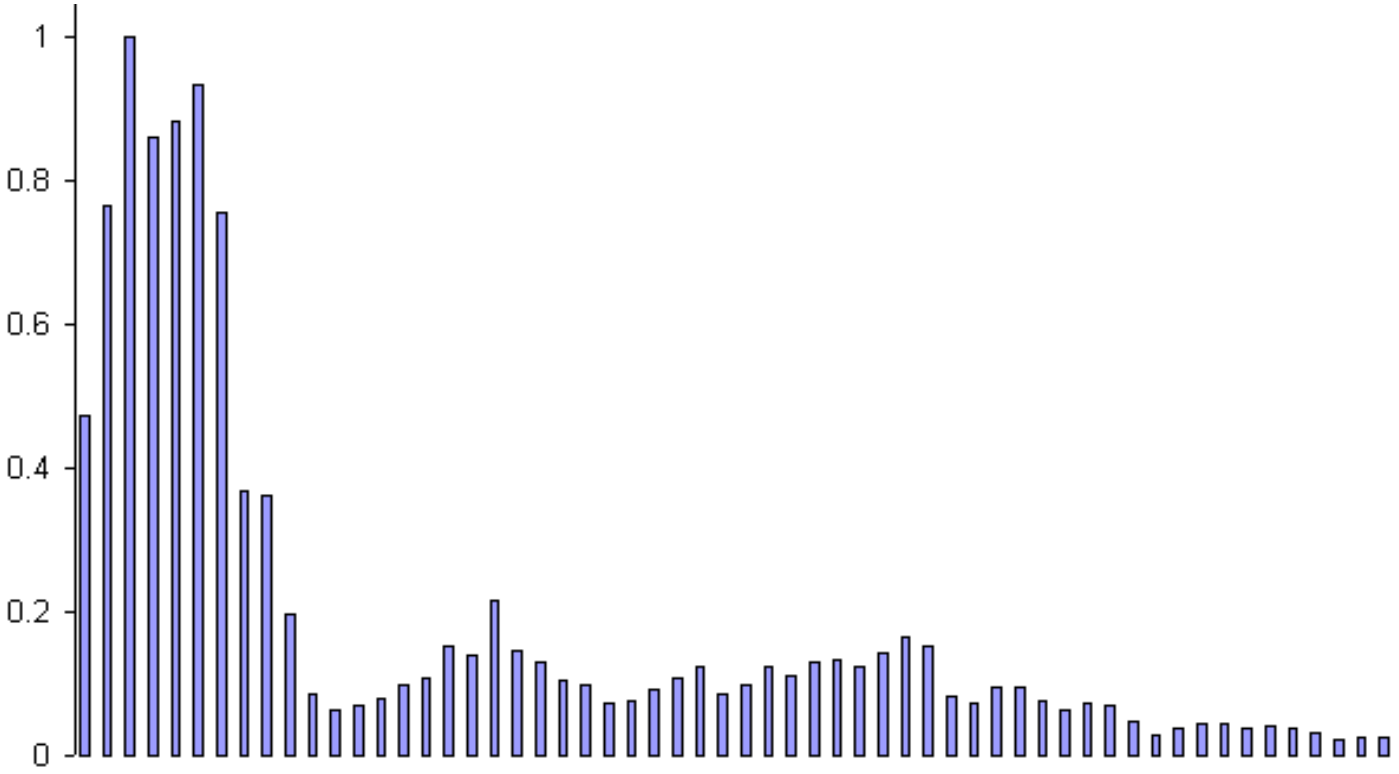


Função sigmóide:



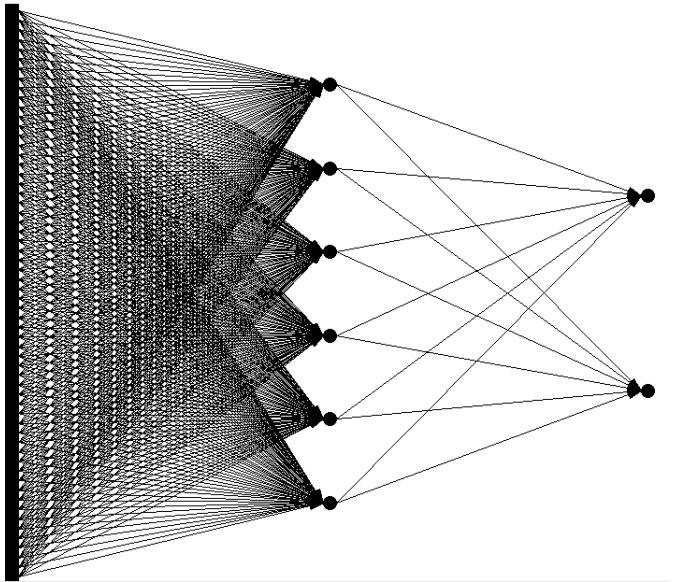
Exemplo: Reconhecimento Voz

- Tarefa: Aprender a discriminar duas pessoas falando “Hello”
- Dados
  - Fonte
    - Steve
    - David
  - Formato
    - Distribuição de Frequencia (60)



Exemplo: Reconhecimento Voz

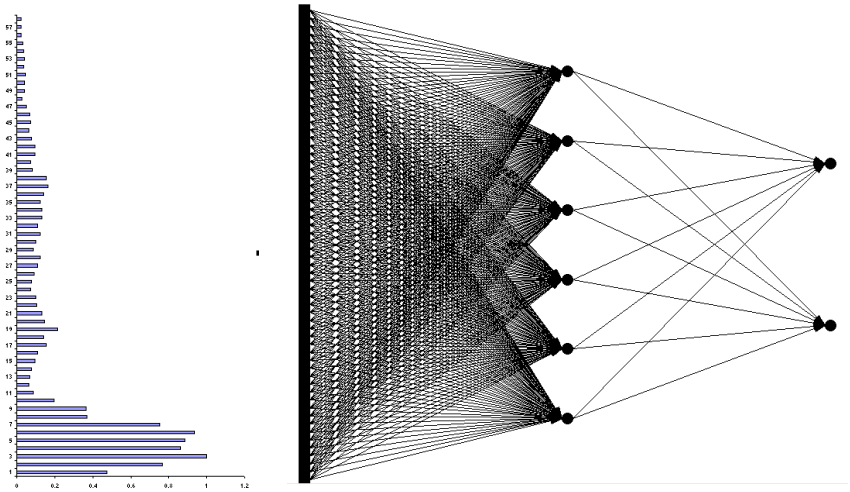
- Arquitetura da Rede
  - Rede Feed Forward
    - 60 entradas (uma para cada frequência)
    - 6 neuronios na camada escondida
    - 2 saidas (0-1 ´para “Steve”, 1-0 para “David”)



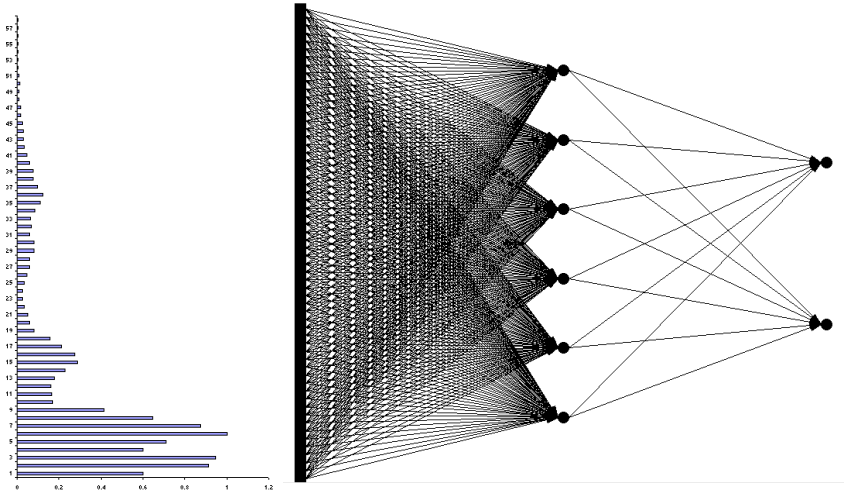
Exemplo: Reconhecimento Voz

Apresentando os dados

Steve



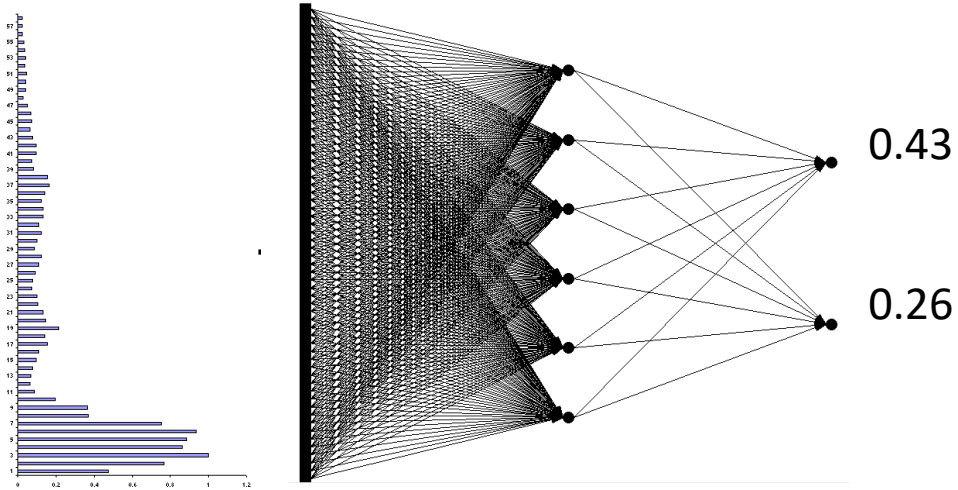
David



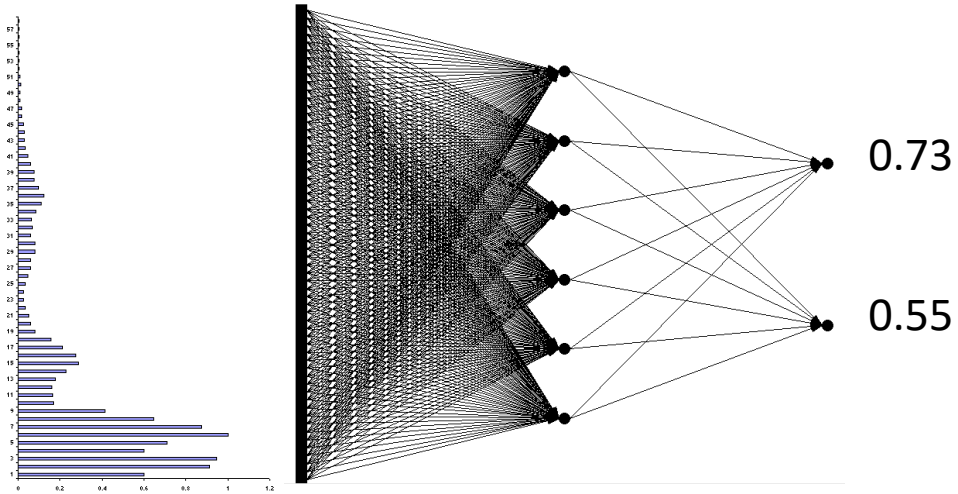
Exemplo: Reconhecimento Voz

Apresentando os dados (Rede não-Treinada)

Steve



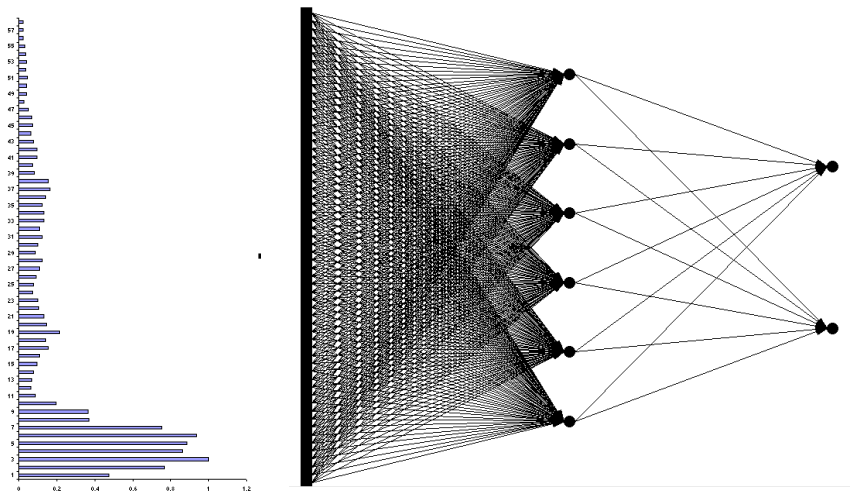
David



Exemplo: Reconhecimento Voz

Calculando os erros

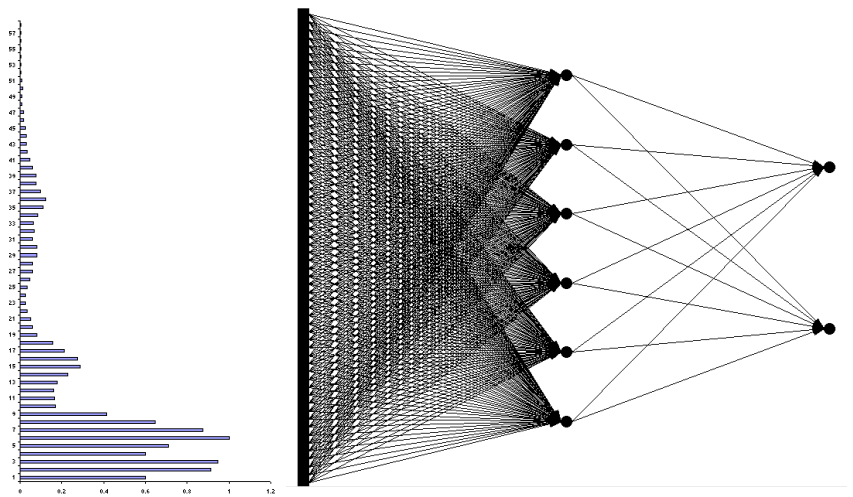
Steve



$|0.43 - 0| = 0.43$

$|0.26 - 1| = 0.74$

David



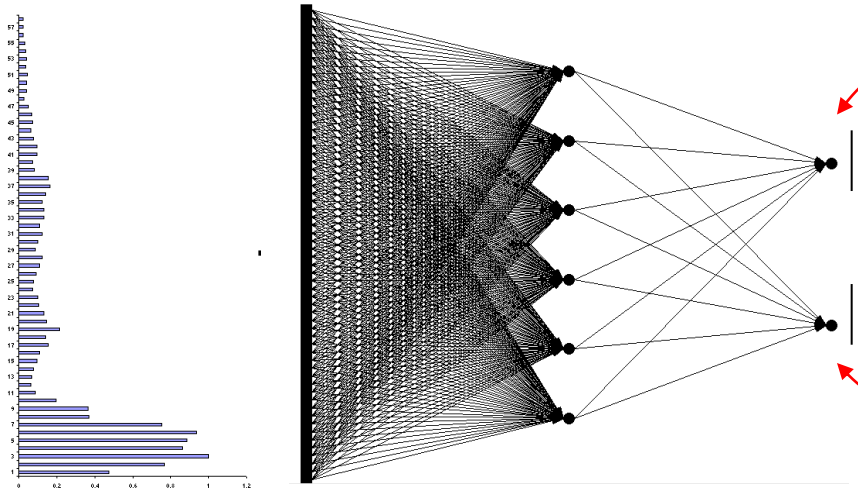
$|0.73 - 1| = 0.27$

$|0.55 - 0| = 0.55$

Exemplo: Reconhecimento Voz

Ajustando os pesos

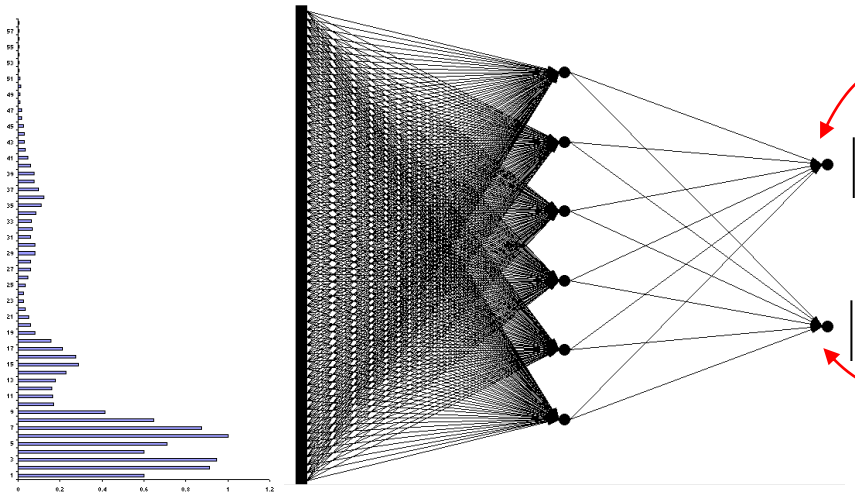
Steve



$|0.43 - 0| = 0.43$

$|0.26 - 1| = 0.74$

David



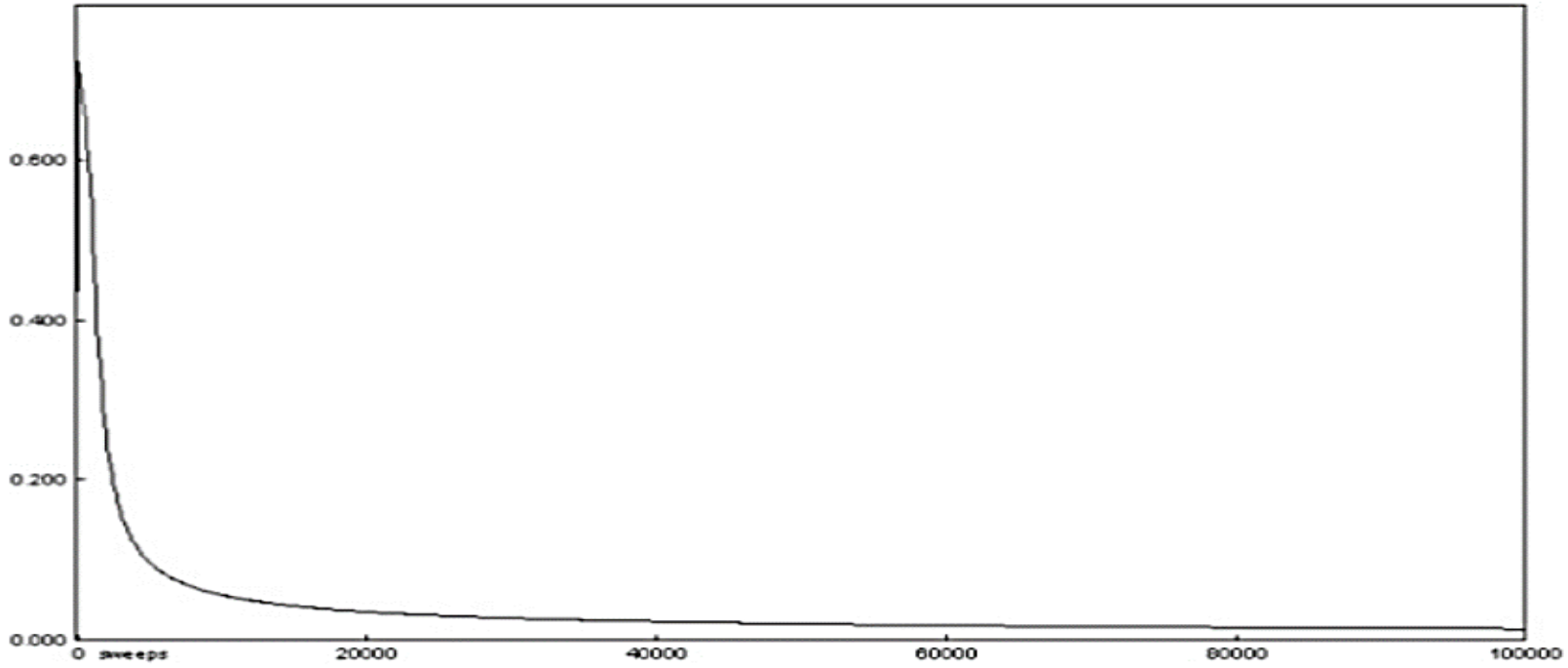
$|0.73 - 1| = 0.27$

$|0.55 - 0| = 0.55$

Exemplo: Reconhecimento Voz

- Repete processo para todos os pares de treinamento
  - Apresenta os dados → Calcula o erro → Ajusta os pesos

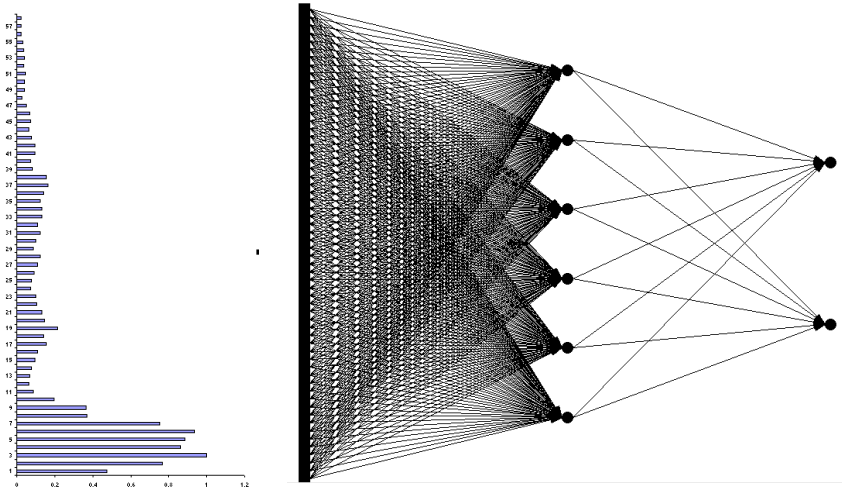
- Repetindo o procedimento várias vezes…o erro vai diminuindo a rede vai aprendendo



Exemplo: Reconhecimento Voz

Apresentando dados (rede treinada !)

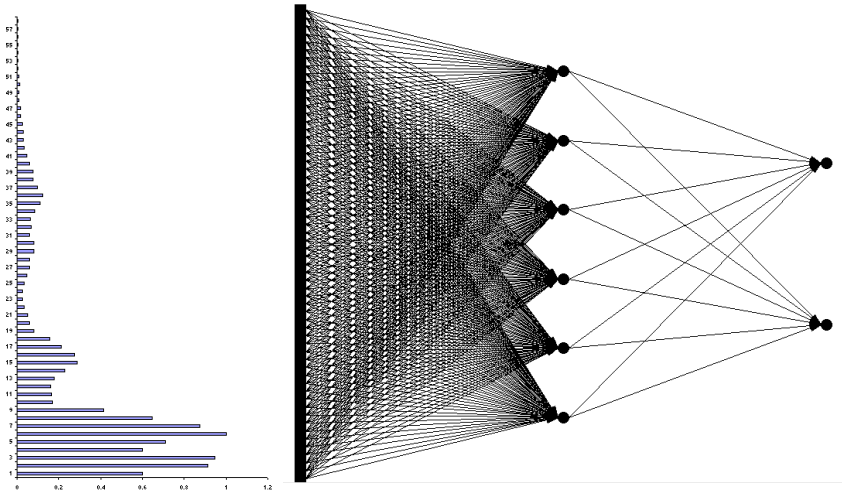
Steve



0.01

0.99

David

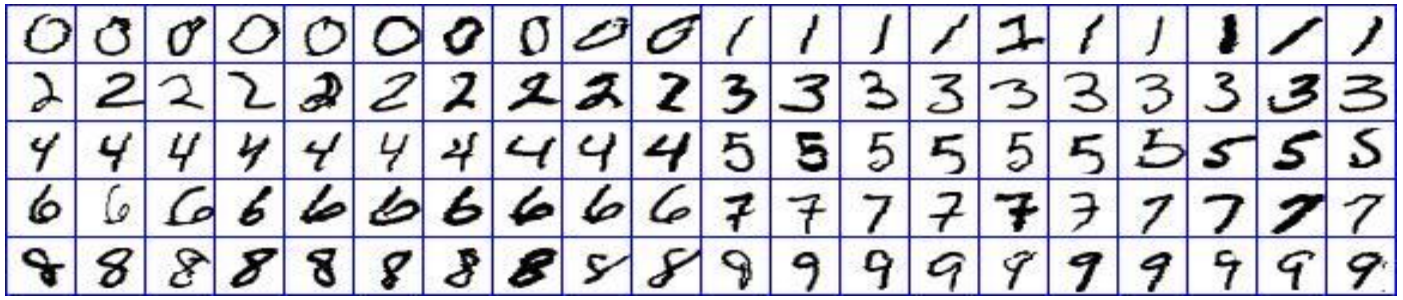


0.99

0.01

Reconhecimento de Dígitos Manuscritos

- Base de 60.000 dígitos manuscritos identificados
  - THE MNIST DATABASE of handwritten digits
  - Google Labs & NYU (New York Univ.)

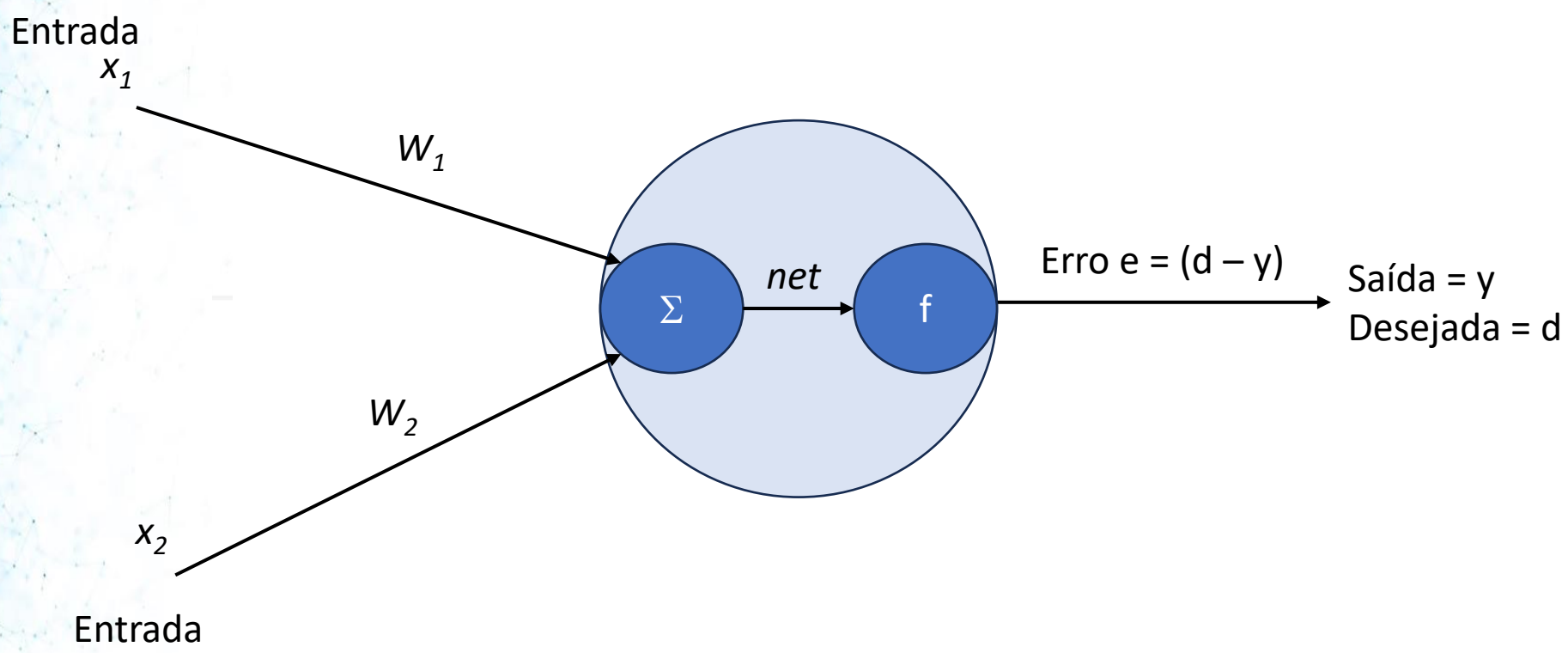


- Rede Neural Projetada com:
  - 400 neurônios de Entrada (imagem 20x20)
  - 10 neurônios de saída (1 por dígito)
  - 300 neurônios em camada escondida
  - Interconexões completas entre camadas
  - Alcançou, após 7 dias de treinamento, uma taxa de erro de1,6%

Ajustando os pesos de uma MLP

Para o treinamento de um Perceptron, tínhamos:

$$Wj = Wj + \eta * (d - y) * f'(net) * xj$$



Ajustando os pesos de uma MLP

Para o treinamento de um Perceptron, tínhamos:

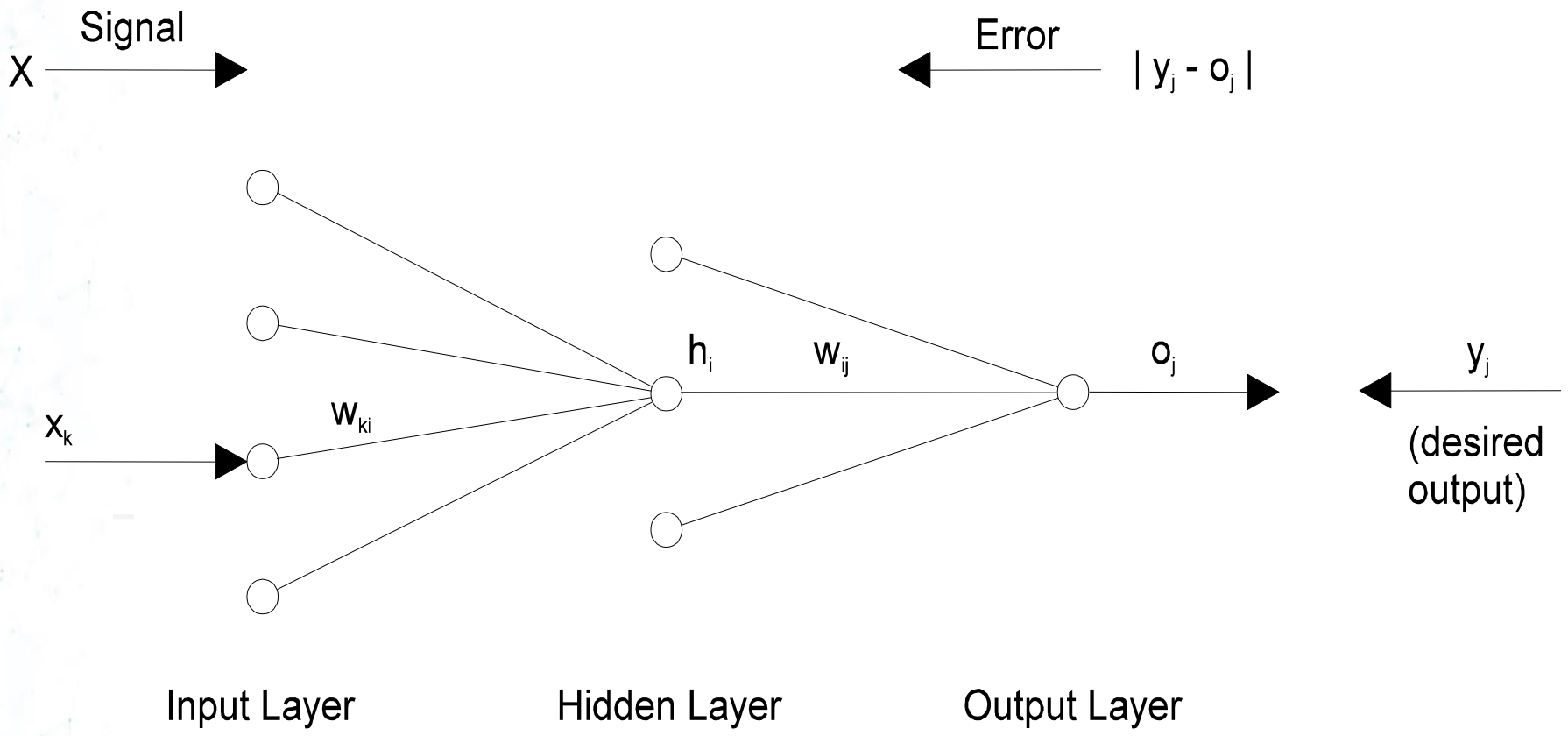
$$Wj = Wj + \eta * (d - y) * f'(net) * xj$$

Mas o perceptron tinha uma camada, e quando temos camadas escondidas, o que fazemos com o valor do erro e de xj ?

Usamos um algoritmo chamado Backpropagation

- *propaga o erro da saída nas camadas interiores*
- *Usa, ao invés de xj (valor de entrada), o valor que veio do neurônio i anterior*

Back Propagation (Retropropagação)



O sinal se propaga para a frente e o erro para trás...

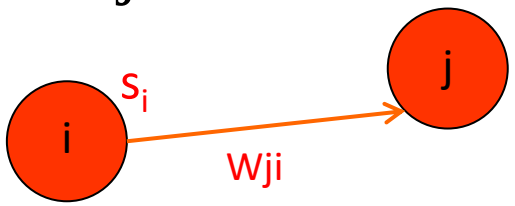
Backpropagation

- Método do Gradiente Descendente
  - O ajuste é feito na direção contrária ao gradiente:

$$\Delta w_{ji} = -\eta \frac{\partial E_p}{\partial w_{ji}} \Rightarrow \Delta w_{ji} = \eta \cdot e_j \cdot s_i$$

A Variação de cada peso depende, para o neurônio j:

- η = taxa de aprendizado $(0 < \eta < 1)$
- e_j = Erro da saída do processador j
- S_i = Valor da entrada vinda do neurônio i anterior



Treinamento de uma MLP

A atualização dos pesos, portanto:

Erro

$$Wj = Wj + \eta * \boxed{ErroPropagado * f'(net)} * s_i$$

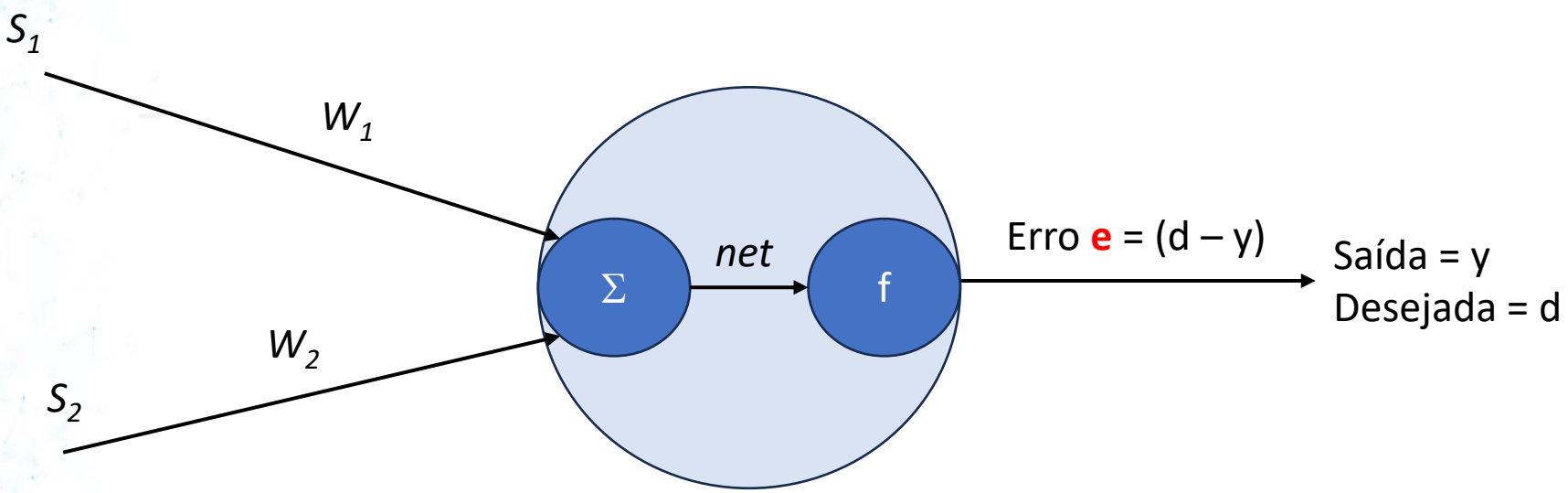
- *ErroPropagado = erro vindo das camadas posteriores*
- *Si = Saída do neurônio i anterior (= Xi se for a camada de entrada)*
- η é uma constante pequena (ex: 0.1), chamada taxa de aprendizado
- *f'(net) para função sigmóide: f(net) (1 – f(net))*

Treinamento de uma MLP

A atualização dos pesos, portanto:

Erro

$$Wj = Wj + \eta * \boxed{ErroPropagado * f'(net)} * s_i$$

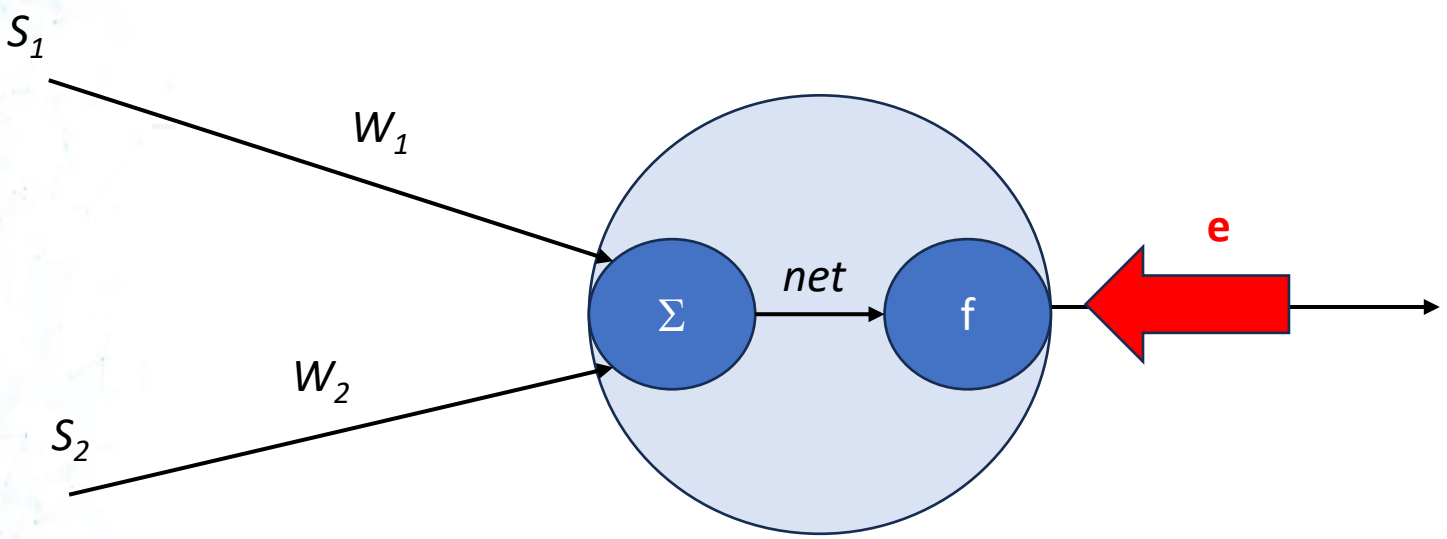


Treinamento de uma MLP

A atualização dos pesos, portanto:

Erro

$$Wj = Wj + \eta * \boxed{ErroPropagado * f'(net)} * s_i$$

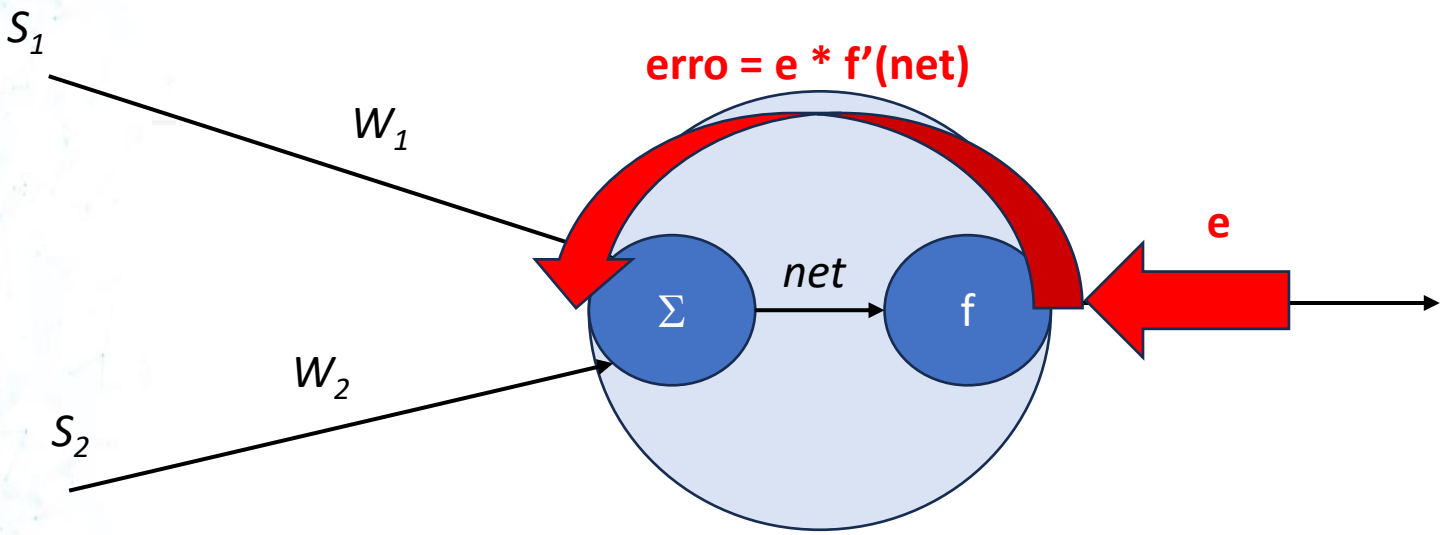


Treinamento de uma MLP

A atualização dos pesos, portanto:

Erro

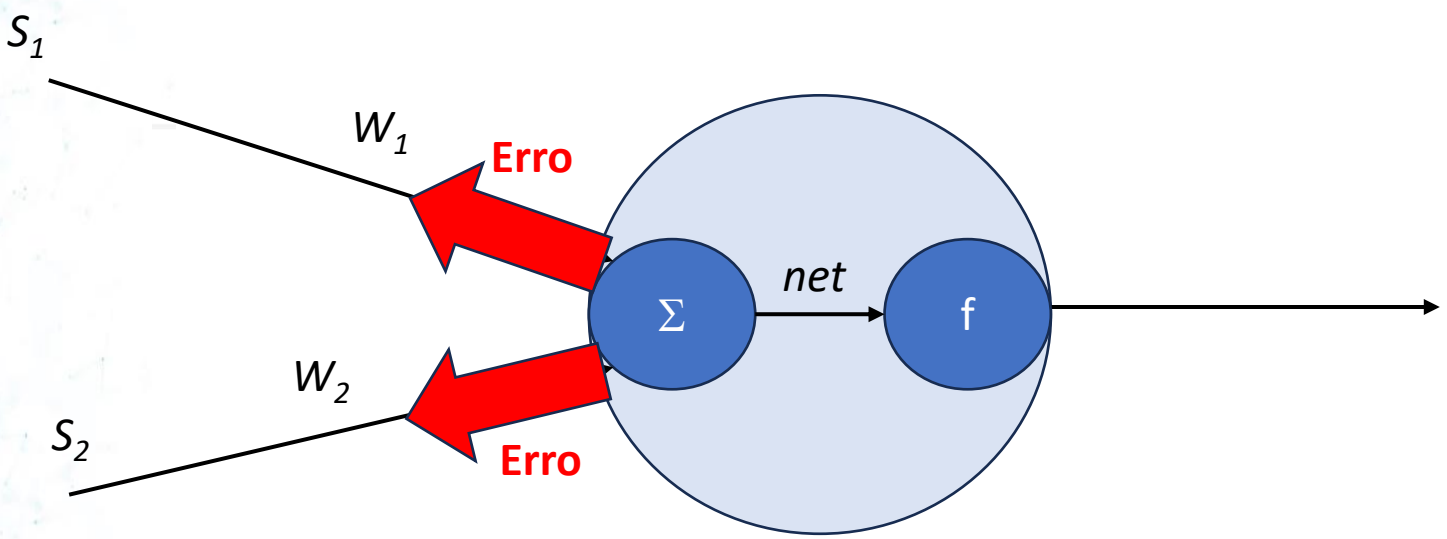
$$Wj = Wj + \eta * \boxed{ErroPropagado * f'(net)} * s_i$$



Treinamento de uma MLP

A atualização dos pesos, portanto:

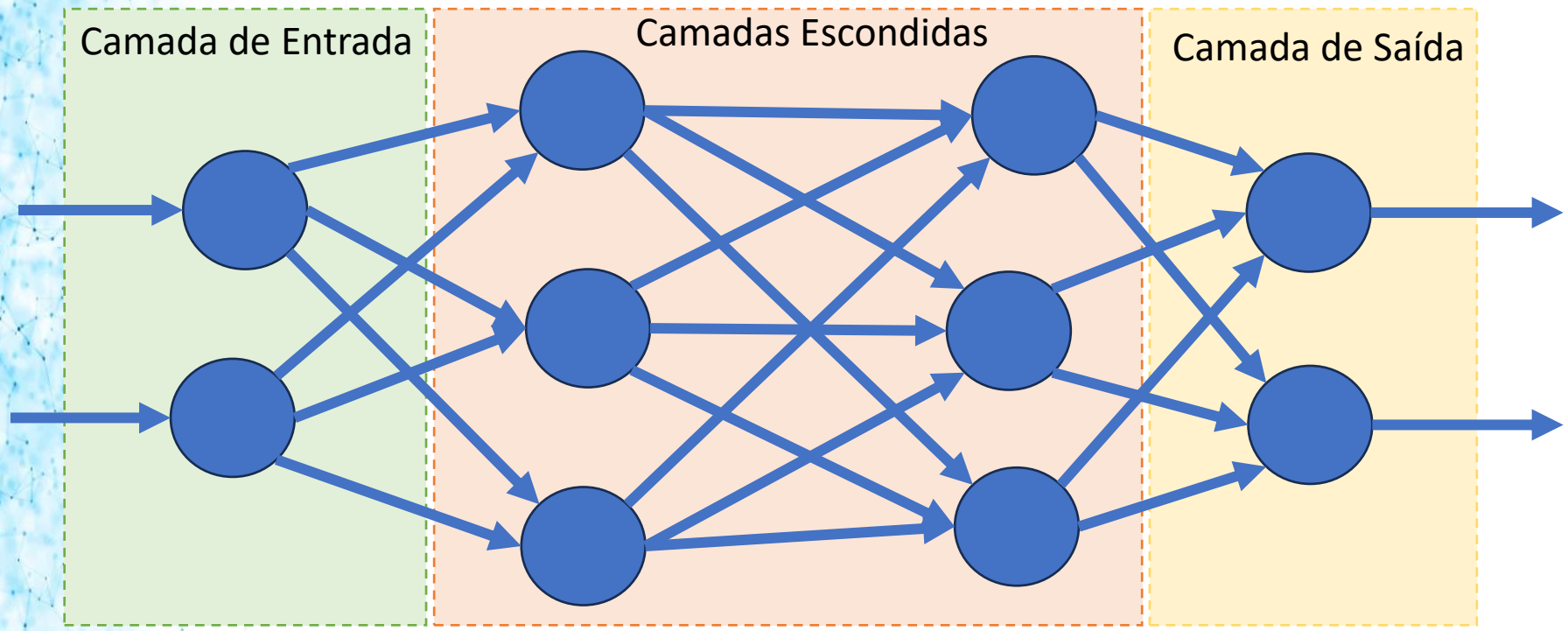
$$Wj = Wj + \eta * \textbf{erro} * s_i$$



Backpropagation

O que é o erro e_j em cada camada ????

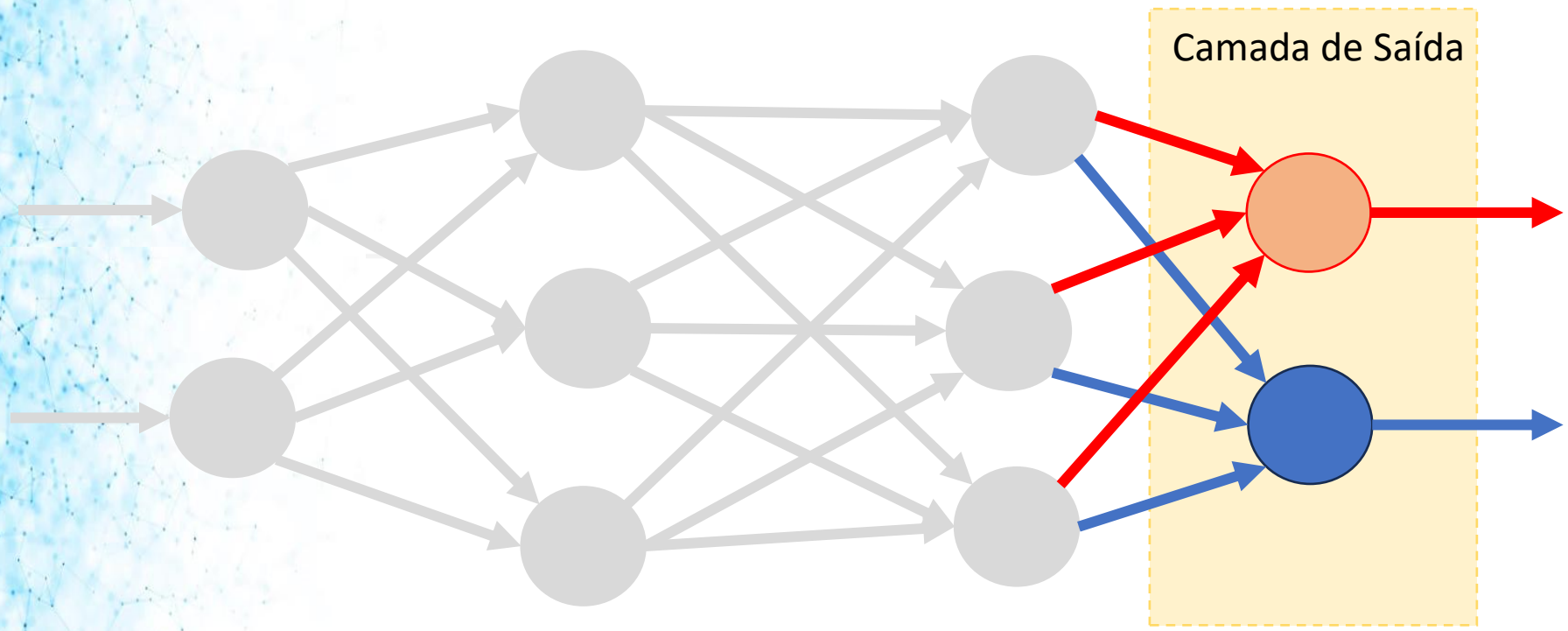
$$\Delta w_{ji} = \eta \cdot e_j \cdot s_i$$



Backpropagation

Camada de Saída

$$\Delta w_{ji} = \eta \cdot e_j \cdot s_i$$



Várias entradas
Uma saída

Backpropagation

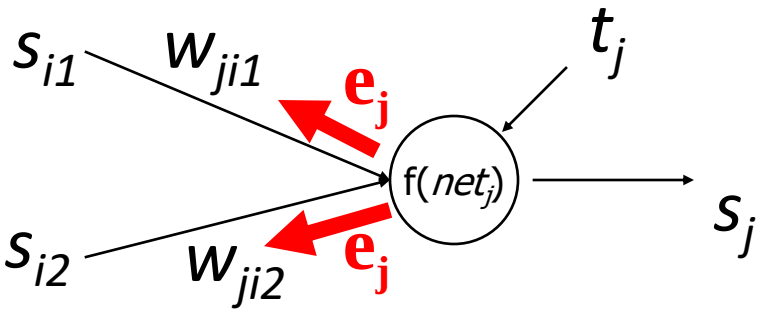
O que é o erro e_j em cada camada ????

$$\boxed{\Delta w_{ji} = \eta \cdot e_j \cdot s_i}$$

Para a **camada de saída** sabemos o valor que deveria dar na saída (t_j) e o valor que efetivamente o neurônio j forneceu (s_j):

$$e_j = (t_j - s_j) * f\,`(net_j)$$

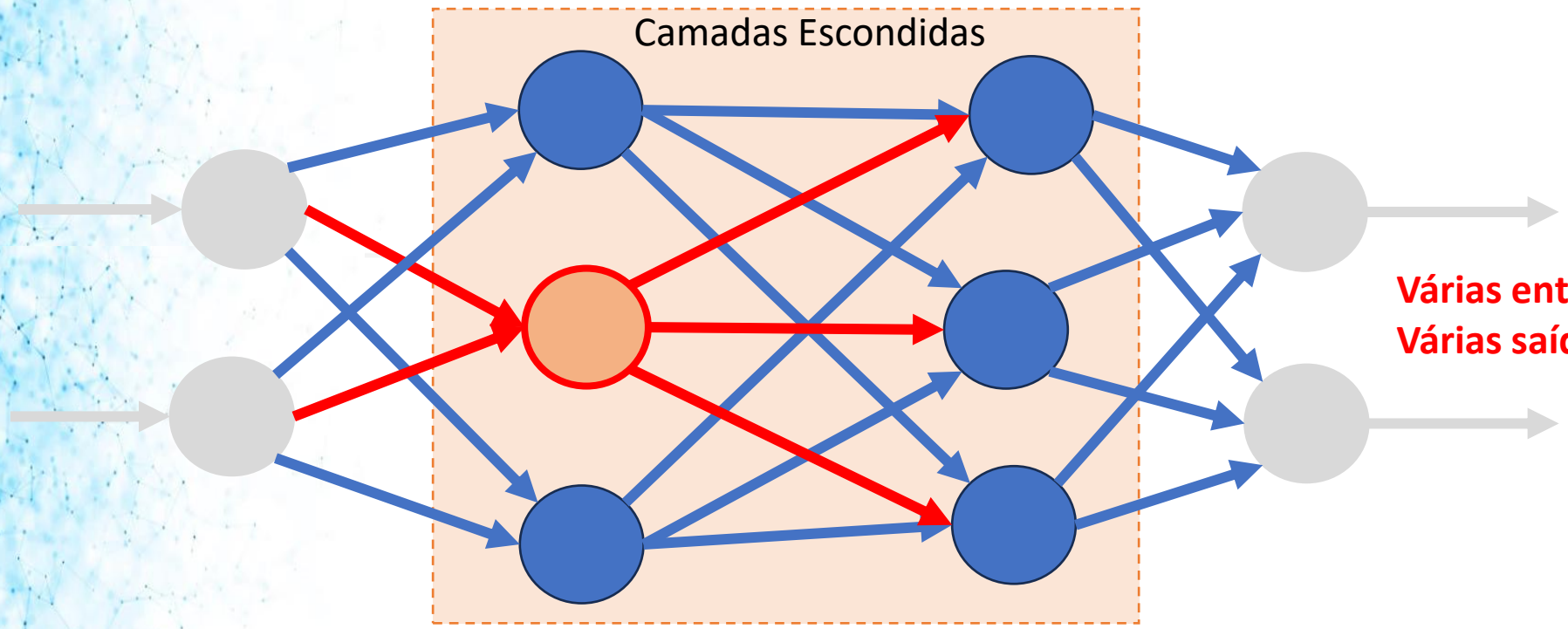
$$\Delta w_{ji} = \eta \cdot \underbrace{(t_j - s_j)}_{d \quad y} f'(net_j) \cdot s_i$$



Backpropagation

E para as camadas escondidas ????

$$\Delta w_{ji} = \eta \cdot e_j \cdot s_i$$



Várias entradas
Várias saídas

Backpropagation

Para as camadas escondidas:

$$\Delta w_{ji} = \eta \cdot e_j \cdot s_i$$

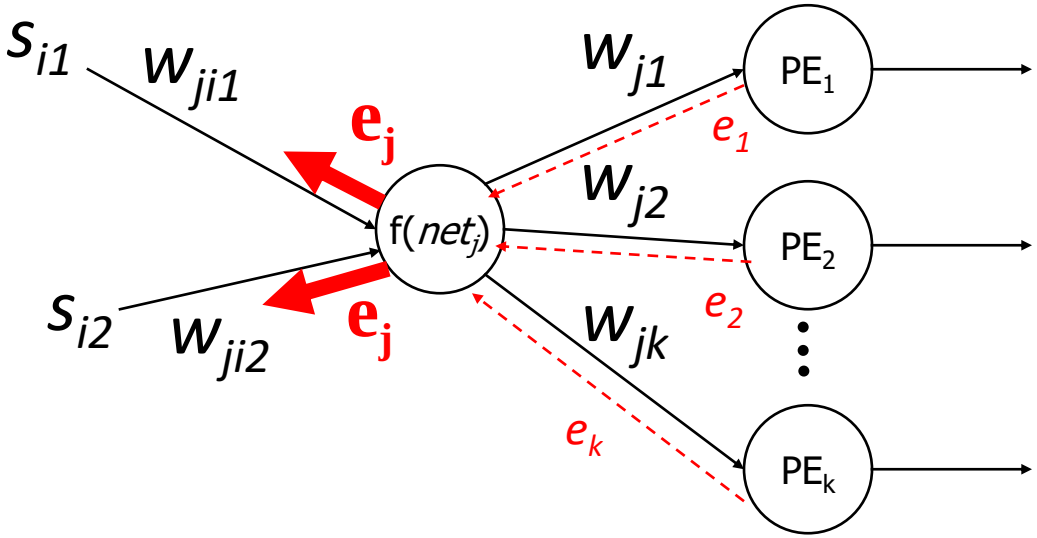
- Conhece-se apenas o erro na camada de saída (e_k)
- Erro na saída (e_k) é função do potencial interno do processador (*neurônio*$_k$)
- O valor do *neurônio* depende dos estados de ativação dos *neurônios* da camada anterior (s_i) e dos pesos das conexões (w_{ki})

Backpropagation

E para as camadas escondidas:

$$\Delta w_{ji} = \eta \cdot e_j \cdot s_i$$

$$e_j = \Sigma_k(e_k * w_{jk}) * f\,`(net_j)$$



$$\Delta w_{ji} = \eta \cdot \left(\sum_k e_k w_{jk}\right) f'(net_j) \cdot s_i$$

Backpropagation

- O que temos portanto:
  - Camada de Saída: $\Delta w_{ji} = \eta \cdot (t_j - s_j) f'(net_j) \cdot s_i$
  - Camada Escondida $\Delta w_{ji} = \eta \cdot (\sum_k e_k w_{jk}) f'(net_j) \cdot s_i$
  - Camada de Entrada:
    - Igual a camada escondida porém Si se torna Xi (valor de entrada)

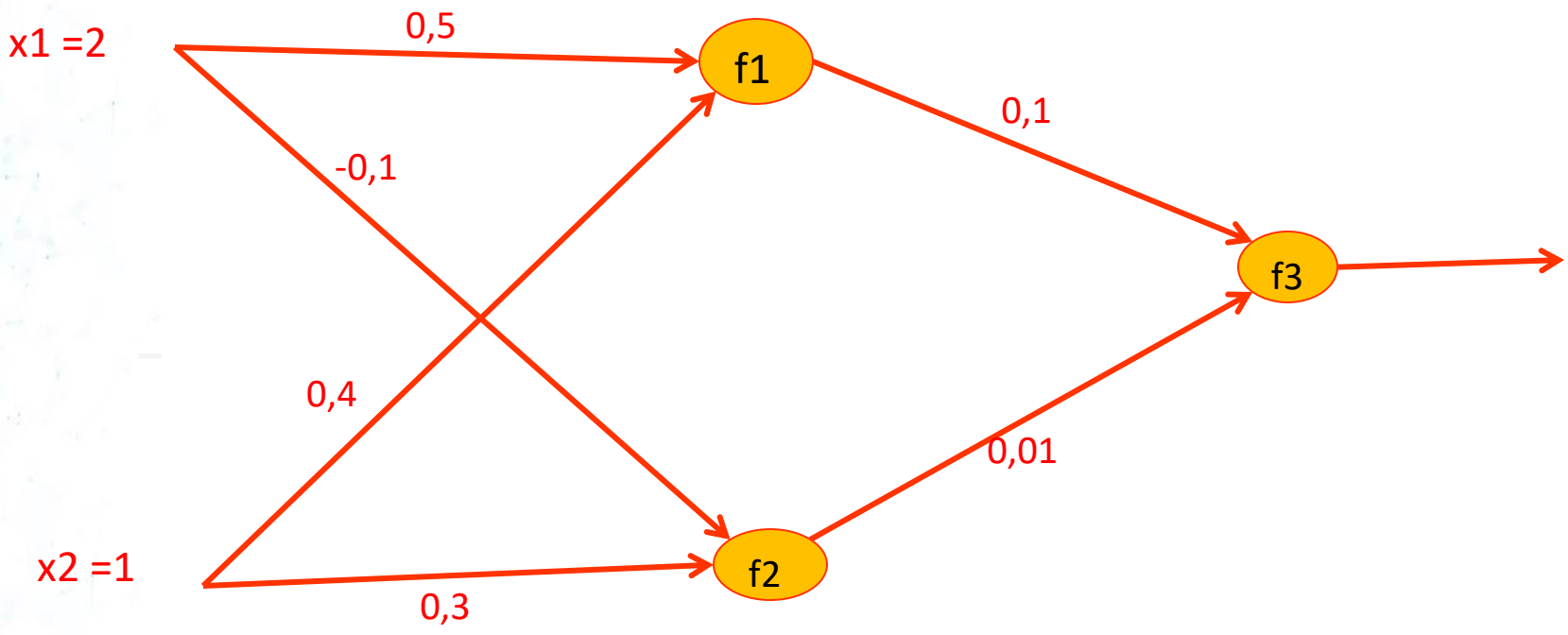
$$\boxed{\Delta w_{in} = \eta \cdot e_i \cdot x_n}$$

- f ` é a derivada da função de ativação f:
  - Se f for Função degrau → f ' (net) = 1
  - Se f for Função sigmóide y → f ' (net) = y' = y (1-y)

Exemplo

- Entradas [x1, x2] = [2,1] ; Saída = 2
- f1 = f2 = f3 = F

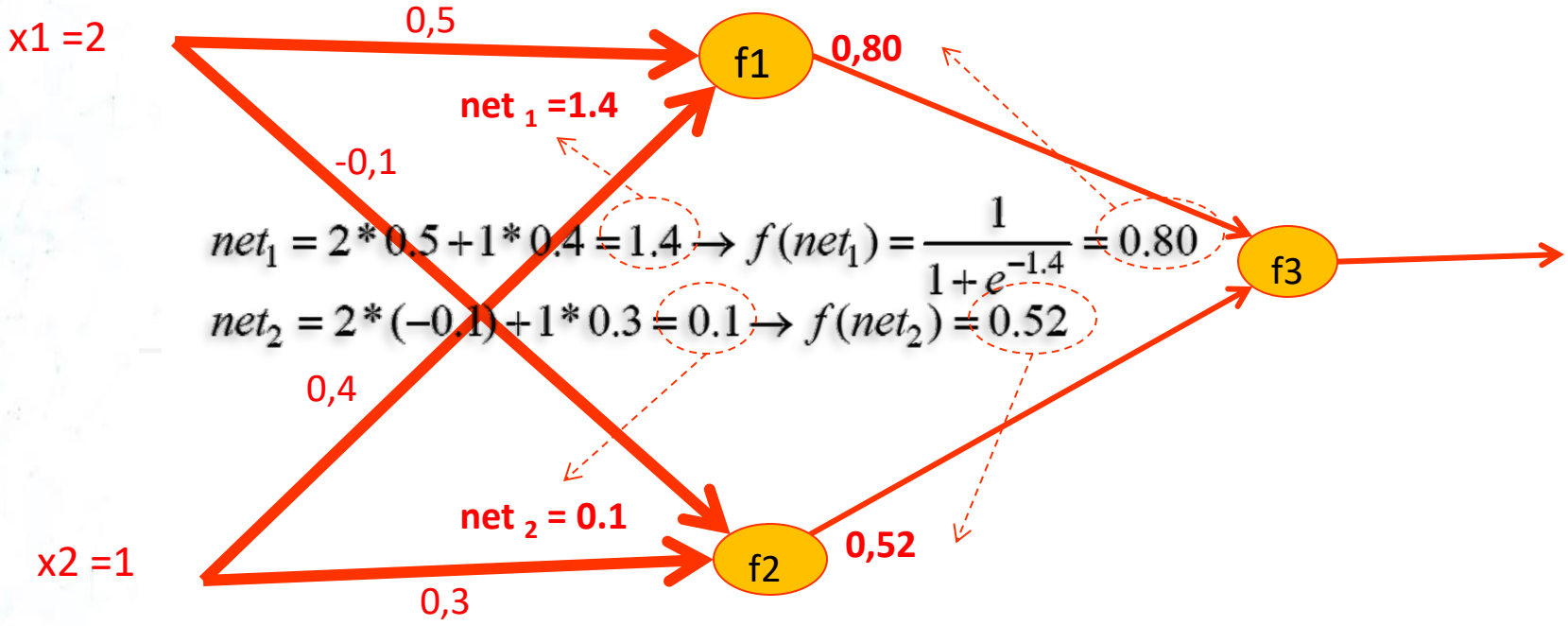
$$F = \frac{1}{(1+e^{-net})} \quad ; net = \Sigma\, x*w$$



Exemplo

- Entradas = [2,1] ; Saída = 2
- Fazendo o RECALL

$$F = \frac{1}{(1+e^{-net})} \quad ; net = \Sigma\, x*w$$



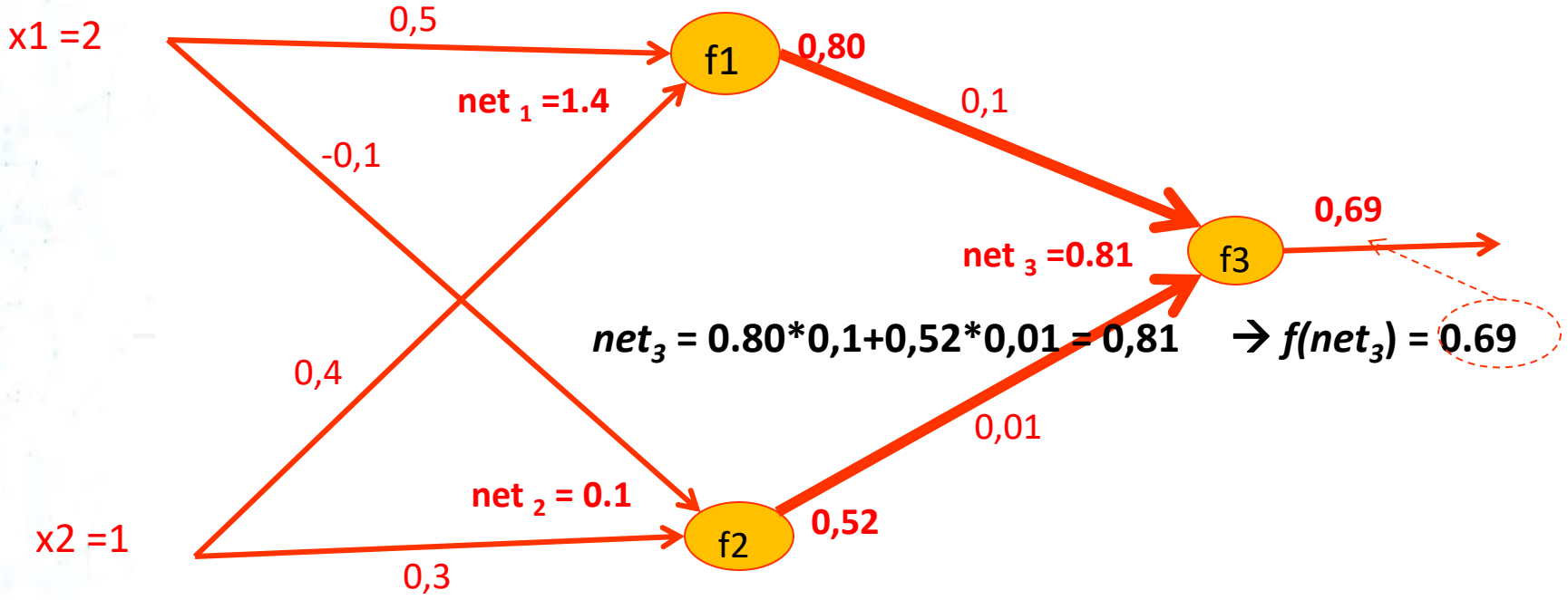
$$net_1 = 2*0.5 + 1*0.4 = 1.4 \rightarrow f(net_1) = \frac{1}{1+e^{-1.4}} = 0.80$$

$$net_2 = 2*(-0.1) + 1*0.3 = 0.1 \rightarrow f(net_2) = 0.52$$

Exemplo

- Entradas = [2,1] ; Saída = 2
- Recall

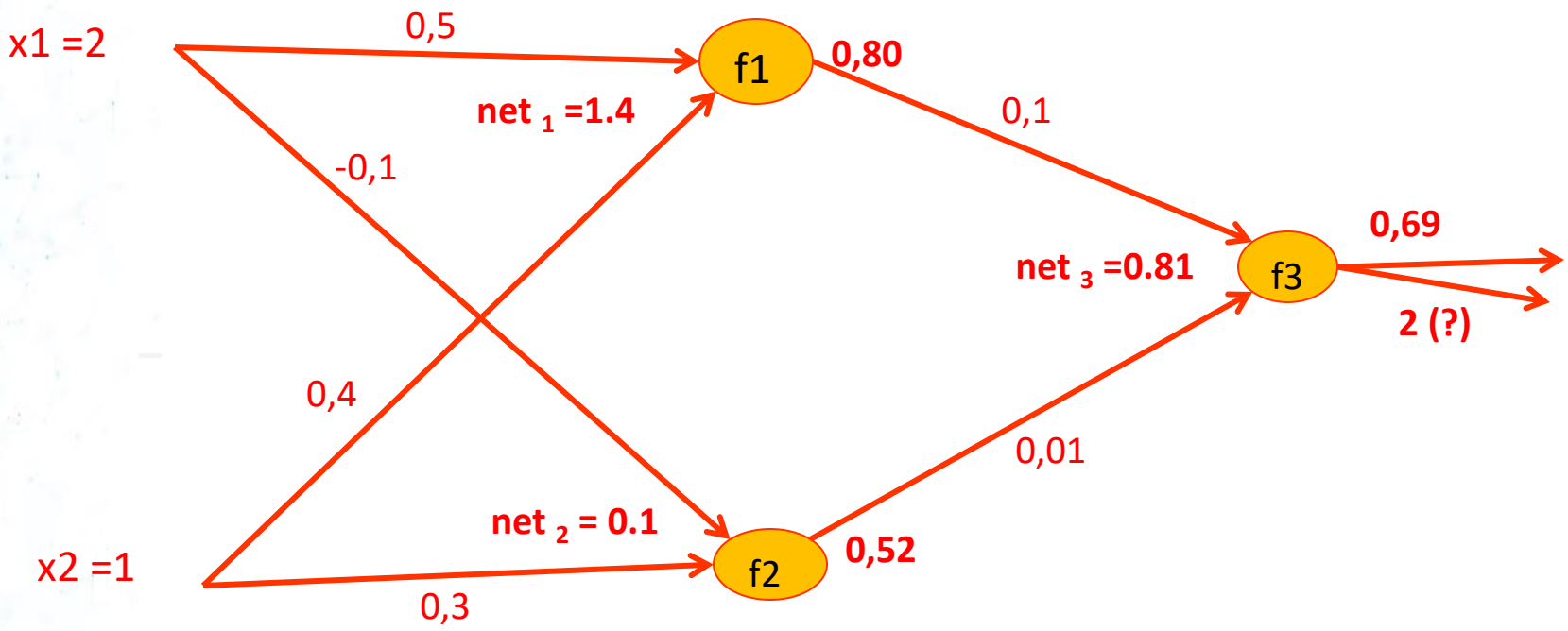
$$F = \frac{1}{(1+e^{-net})} \quad ; net = \Sigma\, x*w$$



net_3 = 0.80*0,1+0,52*0,01 = 0,81 → $f(net_3)$ = 0.69

Exemplo

- Entradas = [2,1] ; Saída = 2



Exemplo

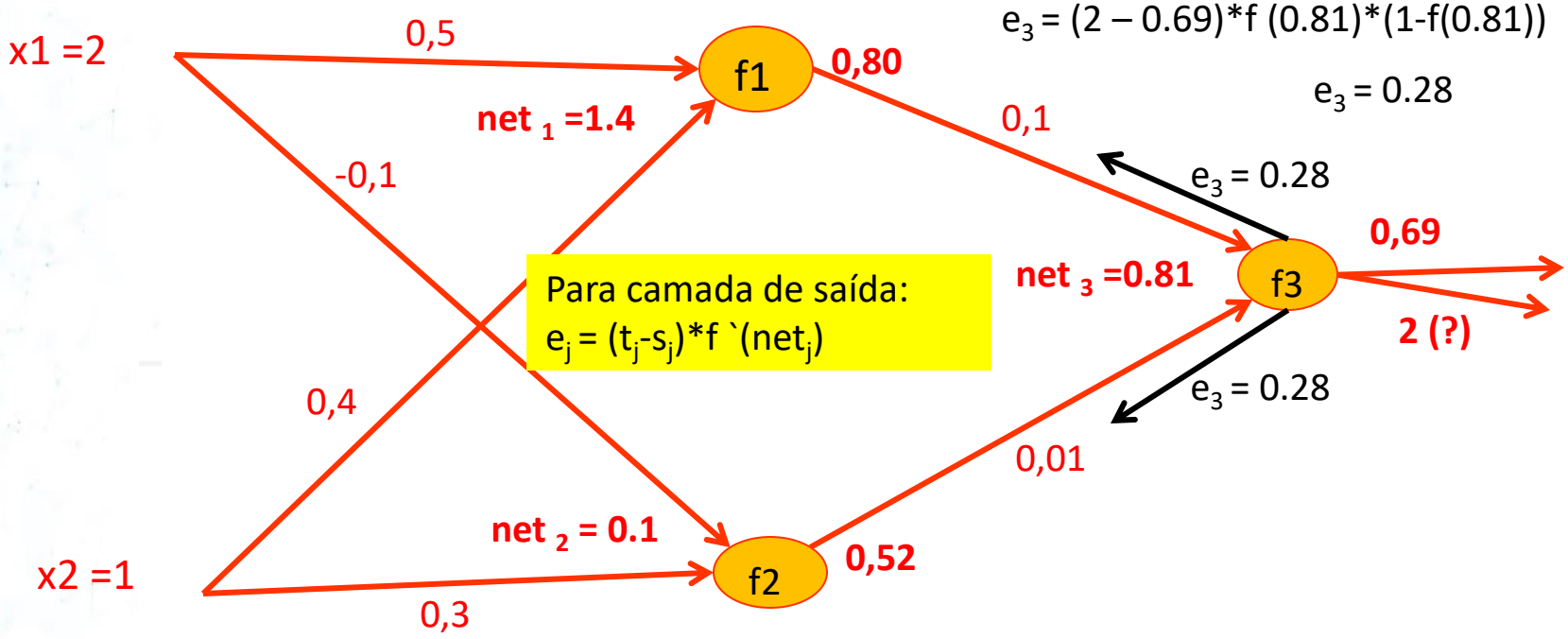
- Entradas = [2,1] ; Saída = 2
- Backpropagation com η =0.4

$f'(net) = y' = y(1-y)$

$e_3 = (2-0.69)*f`(net3)$

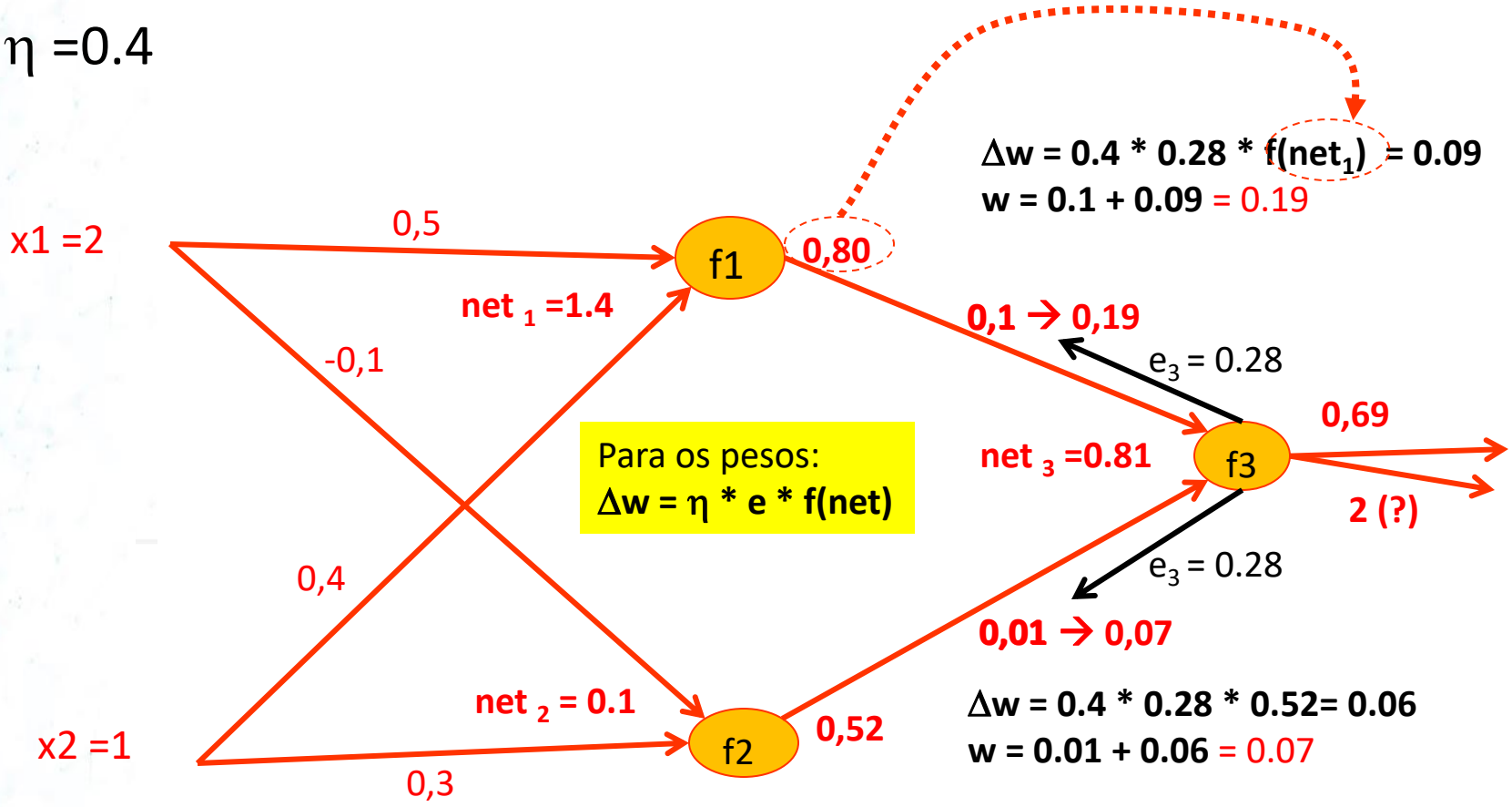
$e_3 = (2 - 0.69)*f(0.81)*(1-f(0.81))$

$e_3 = 0.28$



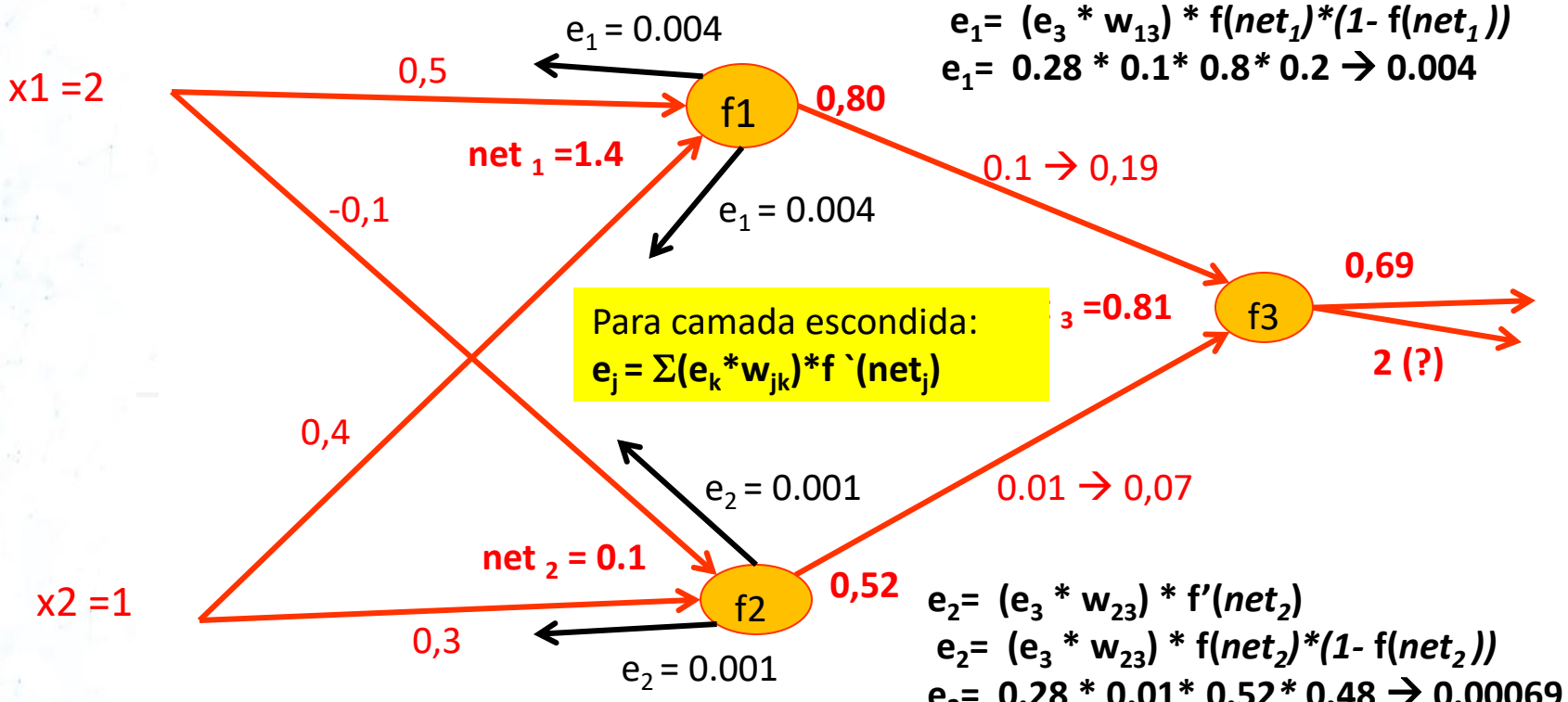
Exemplo

- Entradas = [2,1] ; Saída = 2
- η =0.4



Exemplo

- Entradas = [2,1] ; Saída = 2
- η =0.4



$e_1 = (e_3 * w_{13}) * f'(net_1)$

$e_1 = (e_3 * w_{13}) * f(net_1)*(1- f(net_1))$

$e_1 = 0.28 * 0.1 * 0.8 * 0.2 \rightarrow 0.004$

$e_2 = (e_3 * w_{23}) * f'(net_2)$

$e_2 = (e_3 * w_{23}) * f(net_2)*(1- f(net_2))$

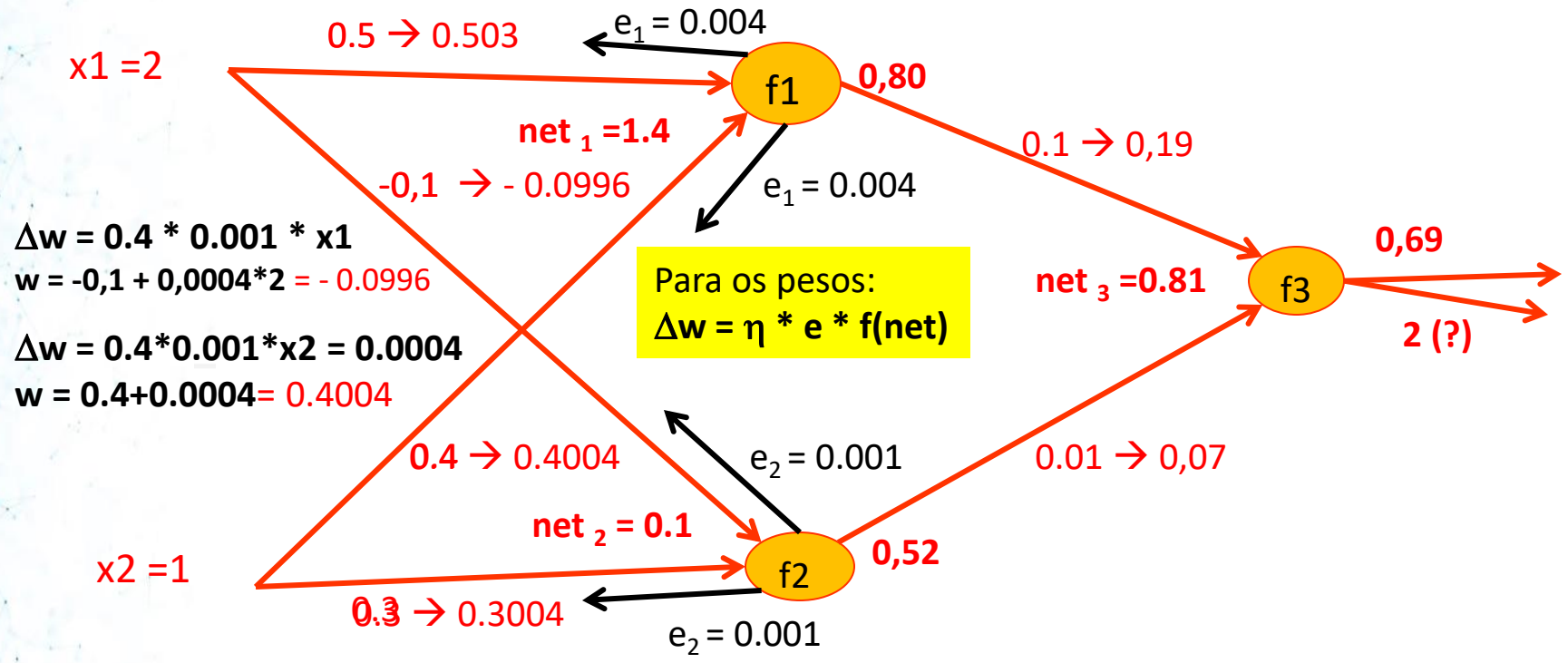
$e_2 = 0.28 * 0.01 * 0.52 * 0.48 \rightarrow 0.00069$

$= 0.001$

Exemplo

- Entradas = [2,1] ; Saída = 2
- η =0.4

Δw = 0.4 * 0.004 * x1 = 0.0032 → w = 0,5 + 0,0032 = 0.503



Δw = 0.4 * 0.001 * x1
w = -0,1 + 0,0004*2 = - 0.0996

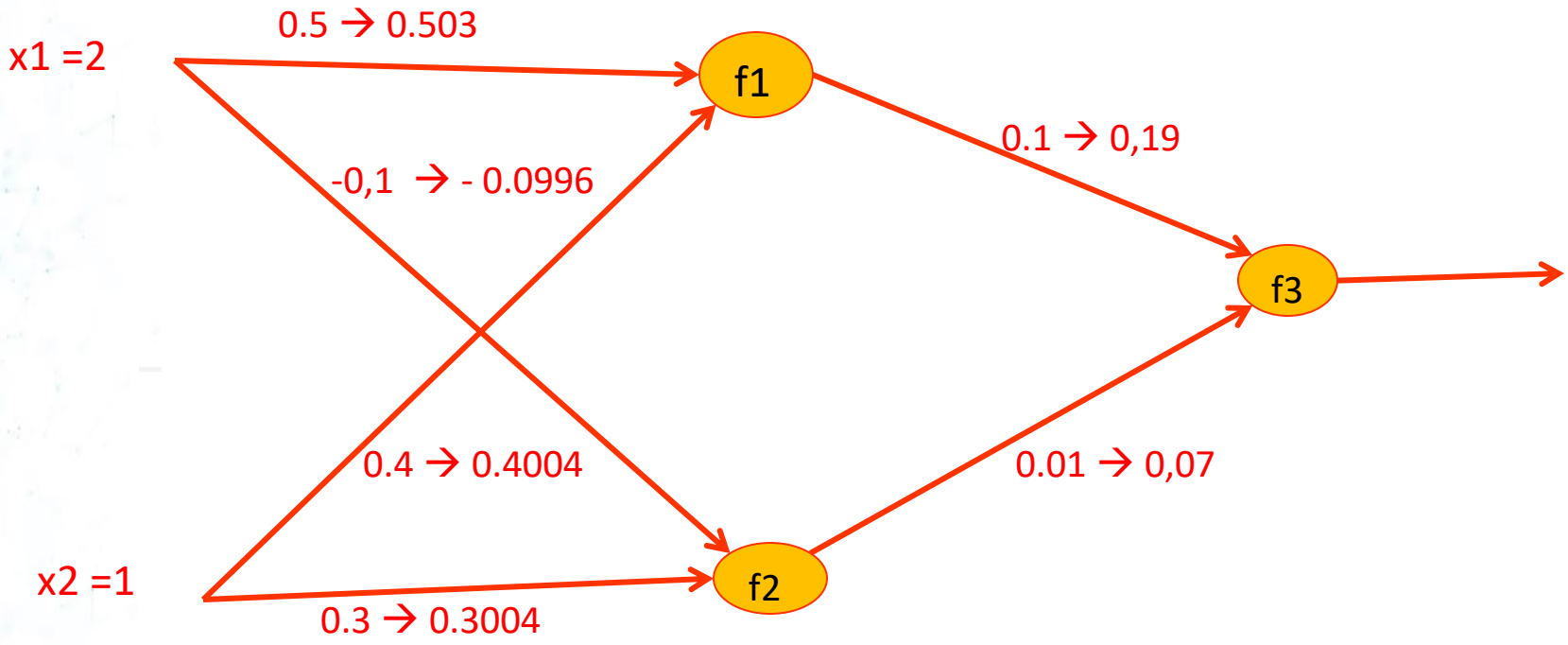
Δw = 0.4*0.001*x2 = 0.0004
w = 0.4+0.0004= 0.4004

Para os pesos:
Δw = η * e * f(net)

Δw = 0.4 * 0.001*x2 = 0.0004 → w = 0,3 + 0.004 = 0.3004

Exemplo

- Entradas = [2,1] ; Saída = 2
- η =0.4



Alguns Problemas em Redes Neurais

- Tamanho da rede neural
  - Não existe nenhum procedimento analítico para definir tamanho da RN
  - Se muito pequena → rede limitada que pode não encontrar os padrões determinados
  - Se muito grande → rede bem flexível, porém pode modelar ruídos e não dar boas respostas a padrões nunca vistos.
- Boas práticas:
  - Não escolha $k_h > 2n$
  - De uma forma geral, $N > (10 \times k_w)$
    - k_h: número de neurônios na camada escondida
    - N: número de padrões ou exemplos de treinamento
    - n: número de variáveis de entrada
    - k_w: quantidade de pesos na RN

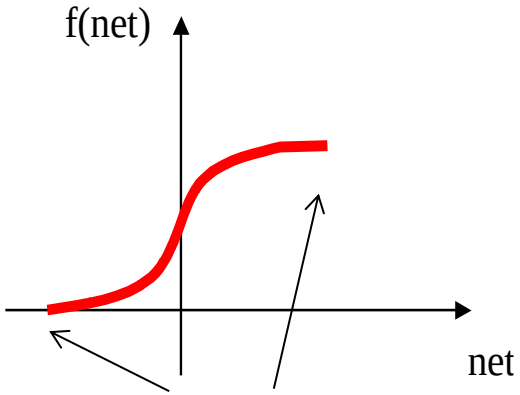
Alguns Problemas

- Paralisia da rede neural
  - Com o treinamento, os pesos podem atingir valores muito grandes, ou seja, *w* >> 0.
  - Com isso, a soma ponderada se torna muito grande

$$\boxed{f'(net) \approx 0 \rightarrow \Delta w_{ji} \approx 0}$$

Os pesos não se atualizam !!

A RN para de aprender !



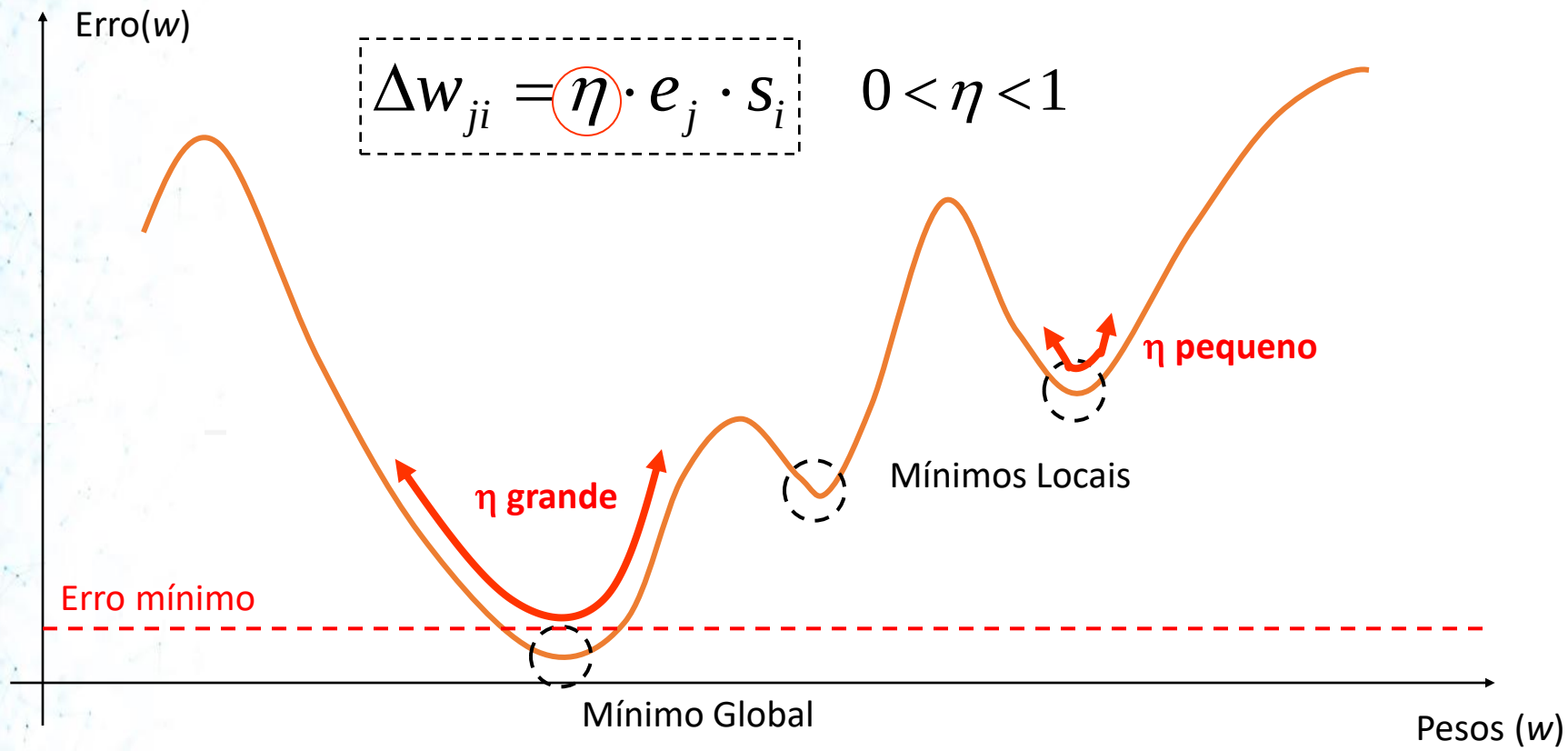
Regiões de derivada zero

- **Boas Práticas:**
  - Deve-se escolher valores de pesos uniformemente distribuídos dentro de um intervalo pequeno
  - O número de processadores na camada escondida deve ser pequeno

Alguns Problemas

Mínimos Locais

$$\Delta w_{ji} = \eta \cdot e_j \cdot s_i \qquad 0 < \eta < 1$$



Alguns Problemas

Mínimos Locais

- Boas Práticas
  - Não usar um η muito pequeno ou usar um η adaptativo
  - Usar o Termo de Momento

$$\Delta w_{ji}(t+1) = \eta \cdot e_j \cdot s_i + \alpha \cdot \Delta w_{ji}(t)$$

- Termo proporcional ao valor anterior de atualização do peso, onde α minimiza as oscilações

$$0 < \alpha < 1$$

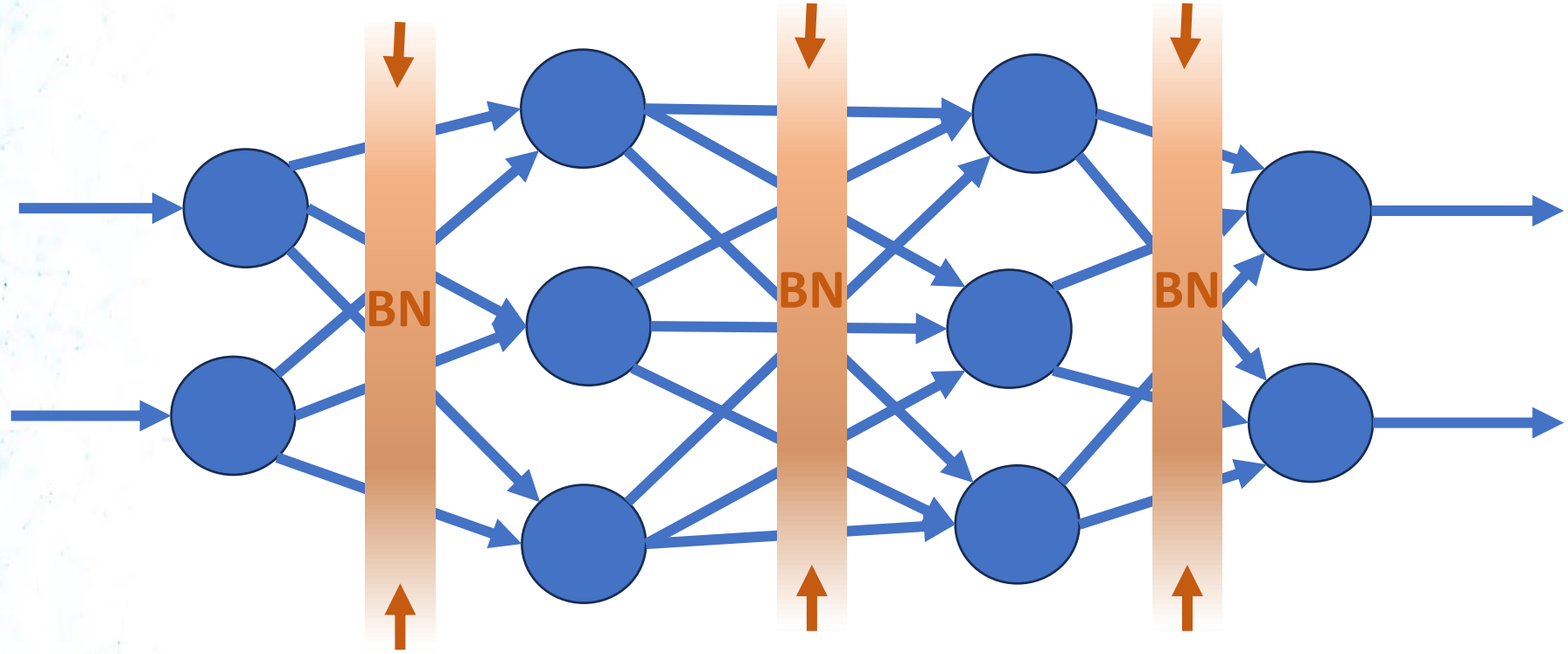
Batch Normalization (BN)

Imagine que você esteja treinando uma rede neural, e os valores das ativações dos neurônios variam muito. Isso pode dificultar o aprendizado porque:

- Os pesos da rede precisam se ajustar a valores muito diferentes a cada iteração.
- O gradiente pode se tornar muito instável (ou muito pequeno), tornando o treinamento mais lento ou até prejudicando a convergência.

Batch Normalization (BN)

BN auxilia na convergência e acelera o aprendizado da RN !!!!!



- Acelera o treinamento, pois reduz variações nas ativações.
- Diminui a dependência da inicialização dos pesos.
- Pode atuar como uma forma de regularização, reduzindo possível overfitting.

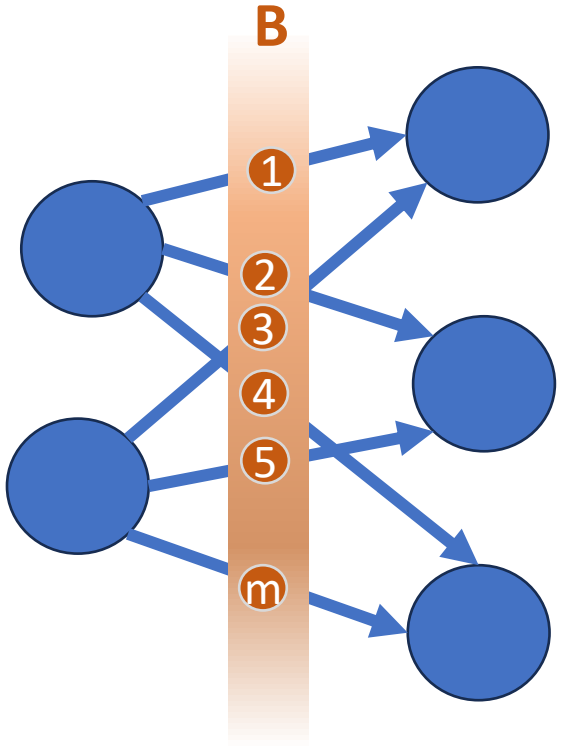
Para cada LAYER B, contendo m valores de ativação, temos:

Cálculo da média: $$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

Cálculo do Desvio-Padrão: $$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

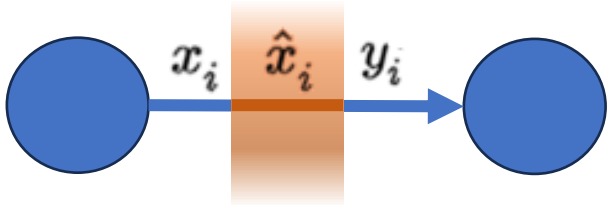
Assim, cada valor de ativação xi é recalculado: $$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\left(\sigma_B\right)^2 + \epsilon}}$$

ε = valor constante pequeno para manter a estabilidade matemática da equação



O Backpropagation atua também nesta camada. E tem dois parâmetros que ele ajusta: γ e β

$$y_i = \gamma\,\hat{x}_i + \beta$$



Código padrão de uma rede neural

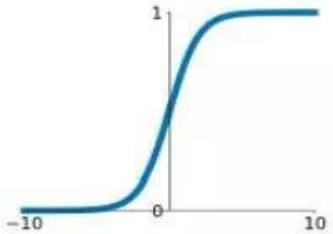
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense, BatchNormalization

model = keras.Sequential([
    Dense(64, activation='relu'),
    BatchNormalization(),
    Dense(32, activation='relu'),
    BatchNormalization(),
    Dense(10, activation='softmax')])
```

Camada de Entrada = 64 neurônios
Camada Escondida = 32 neurônios
Camada de Saída = 10 neurônios

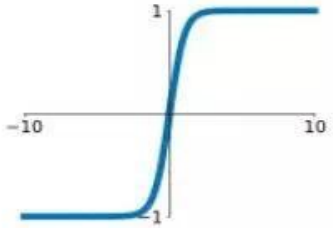
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



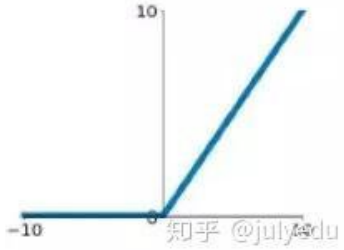
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



Vantagens das Redes Neurais

- Auto-aprendizado: não necessitam de conhecimentos de especialistas para tomar decisões;
- Implementação mais rápida: o tempo necessário para se implementar uma rede é menor que o utilizado para a construção de um sistema especialista equivalente, além do menor custo envolvido.
- Imunidade a falhas: como as unidades da rede operam em paralelo, a destruição ou defeito em um de seus nodos não torna a rede inoperante
- Capacidade de generalização: mesmo com dados incompletos ou imprecisos as redes podem preencher as lacunas sem sofrer degradação;
- Imunidade a ruídos: os dados reais sempre contêm ruído (variações aleatórias adicionadas aos valores originais); as redes conseguem separar o ruído da informação relevante.
- Adaptabilidade: uma vez construída uma rede eficiente em dada aplicação, ela pode sem utilizada em aplicações de tempo-real, sem necessidade de ter sua arquitetura alterada a cada atualização.

Desvantagens das Redes Neurais

- Resultados desconcertantes: as redes podem chegar a conclusões que contrariem as regras e teorias estabelecidas, bem como considerar dados irrelevantes como básicos;
- Treinamento demorado: o treinamento de uma rede, dependendo da aplicação, pode ser demorado (horas ou mesmo dias).
- Caixa-preta: é impossível saber porque a rede chegou a tal conclusão; os milhares de pesos não aceitam interpretação: sabe-se apenas que funcionam.
- Preparação de dados: os dados de entrada necessitam de tratamento prévio: devem ser normalizados, em alguns casos fuzzificados (usando fuzzy logic); dados de má qualidade produzem resultados falhos.
- Trabalho artesanal: não há regras gerais para se determinar o volume de dados de entrada para treinamento, quantas camadas escondidas devem ser utilizadas e a melhor estratégia de treinamento: esses parâmetros só podem ser estabelecidos através de bom-senso, experiência com redes e de tentativa-e-erro.

Avanço das Redes Neurais....

- Nos últimos anos as RN evoluíram para arquiteturas mais avançadas que deram origem:

- CNN – Rede Neural Convolucional (deep-Learning)
- GAN – Generative Adversarial Network
- Entre outras...

- Próxima Aula !

Bibliografia desta aula

Para aprofundamento nos assuntos desta aula, segue a seguinte referência bibliográfica

- Rich, E. (Inteligência Artificial)
  - Capítulo 18
- Russel & Norvig (Artificial Intelligence)
  - Capítulo 19
- Solange Rezende (Sistemas Inteligentes)
  - Capítulo 6

Alguns slides desta aula foram baseados nos slides:

- Anna Reali: “Redes Neurais Artificiais”, Poli-USP.
- Reinaldo Bianchi: “Redes Neurais Artificiais”, FEI-2018.
- Torsten Reil: “Artificial Neural Network”,UK
- Carlos Eduardo Thomaz: Aulas do mestrado em RN. FEI 2010
- Grupo ICA – PUC-Rio
- G5AIAI - Introduction to Artificial Intelligence – University of Nottingham